AMA 564 Deep Learning

2026 Spring

Lecture 4

A Short Recap

Feedforward Neural Networks (Multi-Layer Perceptrons)

- The architecture of a MLP is expressed as a composition of a series of functions

$$f_\theta(x) = \mathcal{A}_L \circ \sigma \circ \mathcal{A}_{L-1} \circ \sigma \circ \cdots \circ \sigma \circ \mathcal{A}_1(x), \quad x \in \mathbb{R}^{d_0}$$

where

$$\mathcal{A}_i(x) = W_i x + b_i, \qquad x \in \mathbb{R}^{d_{i-1}}$$

is the i-th linear transformation with

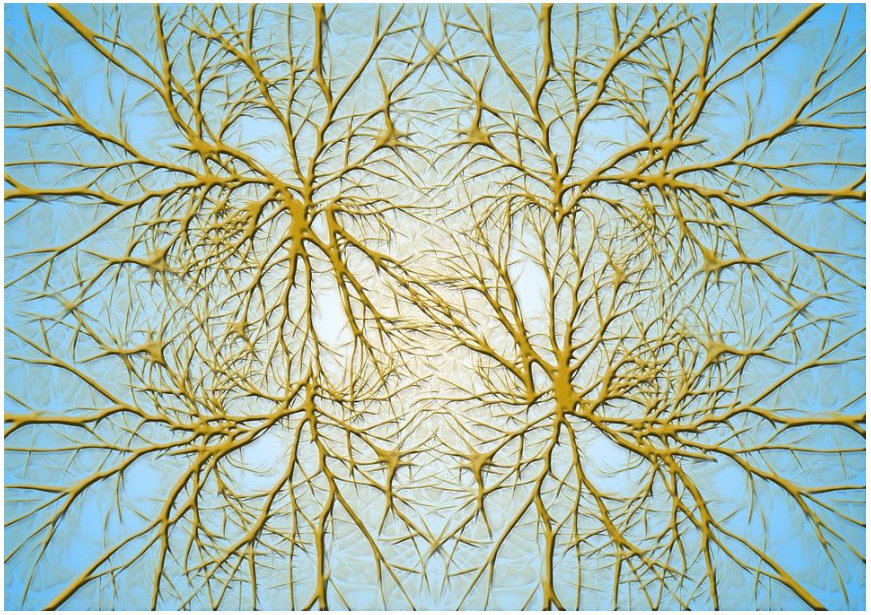
weight matrix $W_i \in \mathbb{R}^{d_i \times d_{i-1}}$ and bias vector $b_i \in \mathbb{R}^{d_i}$,

and σ is the activation function,

e.g., ReLU $\sigma(x) = \max\{x, 0\}$. Sigmoid $\sigma(x) = (1 + e^{-x})^{-1}$.

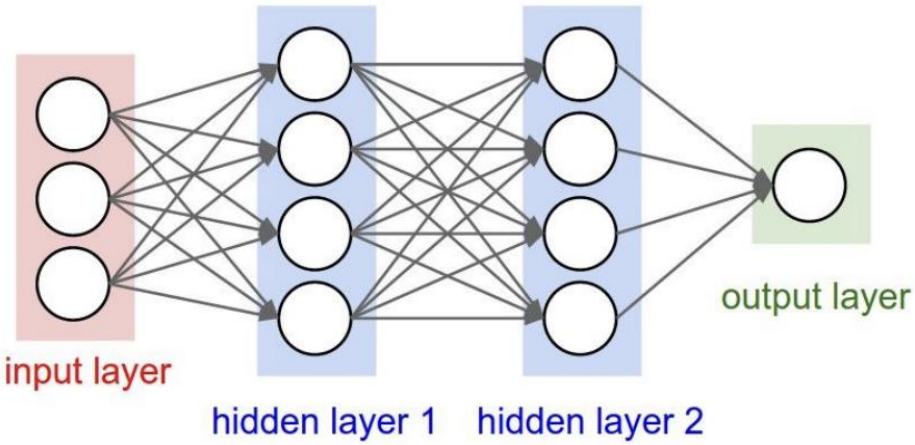
Recap

Biological Neurons



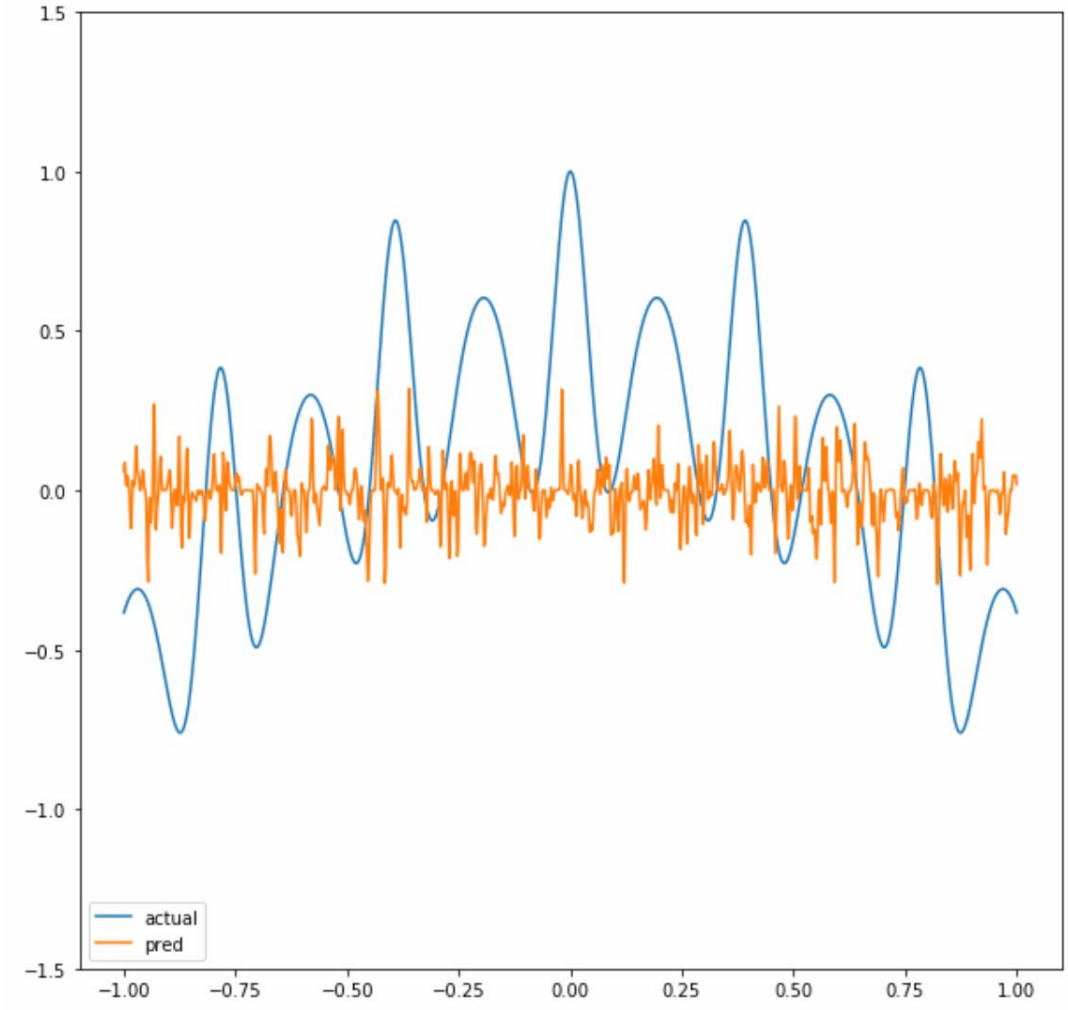
Complex connectivity

Neurons in a neural network:



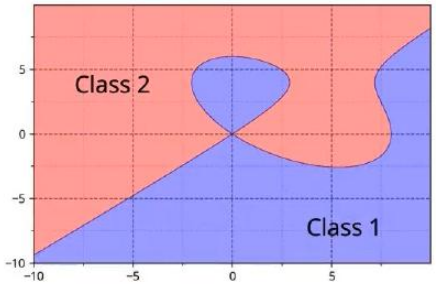
Layer structure
Fully connected
Computational efficient

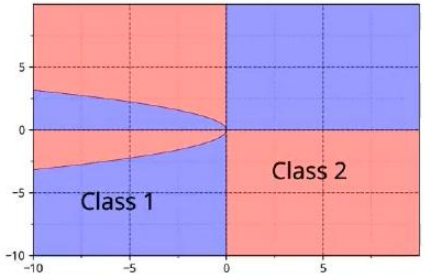
Universality of Neural Networks

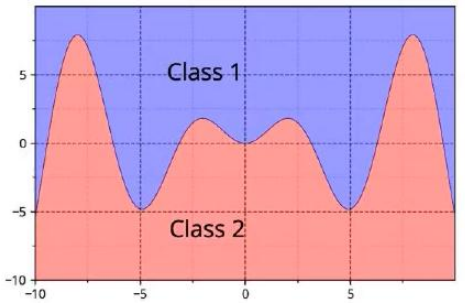


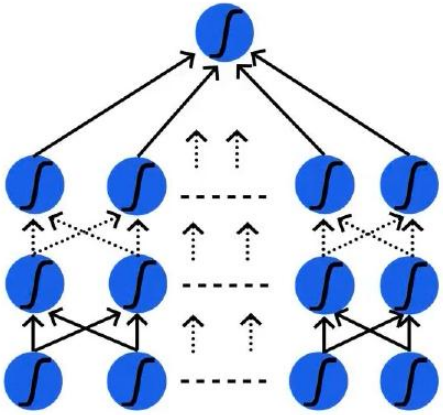
Neural networks can approximate regression functions very well.

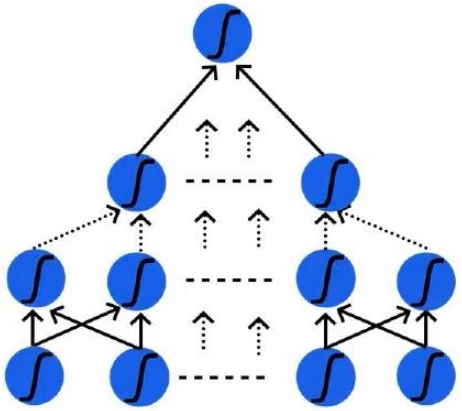
Universality of Neural Networks

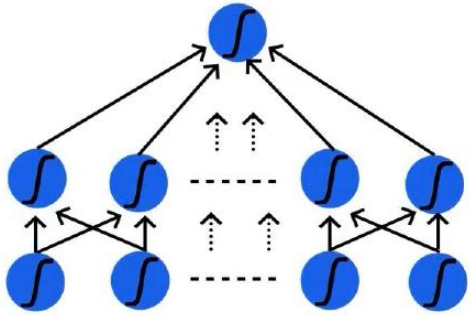








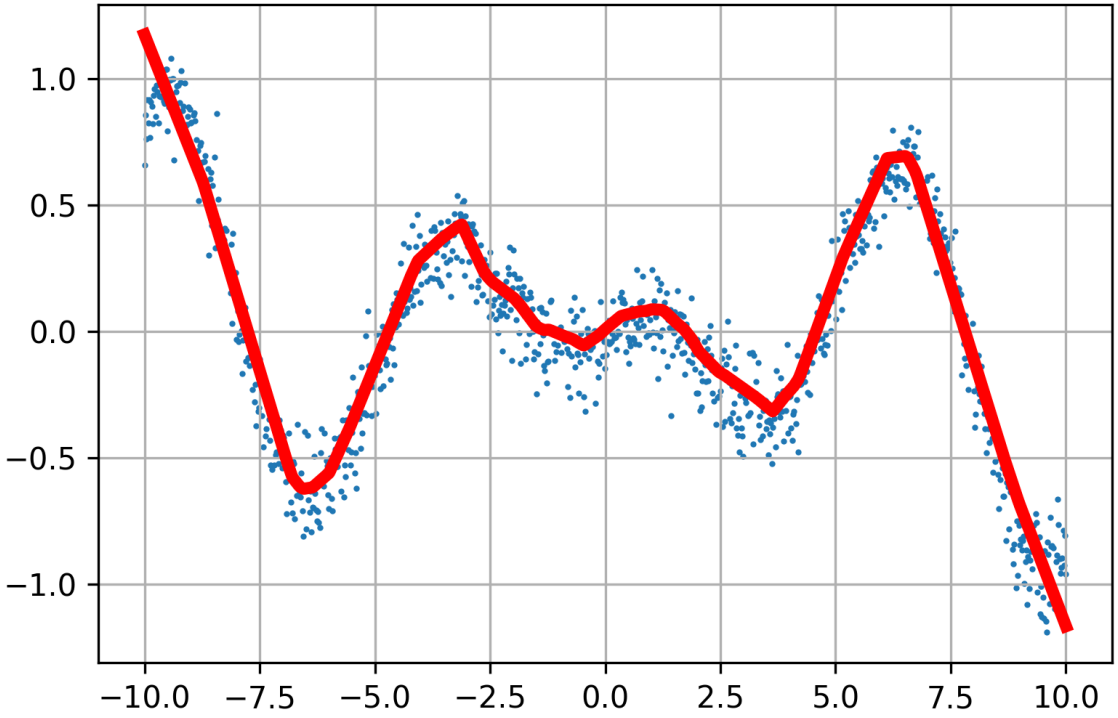




Neural networks can also approximate classification regions very well.

Revisit: Deep Nonparametric Regression

Recall the regression problem



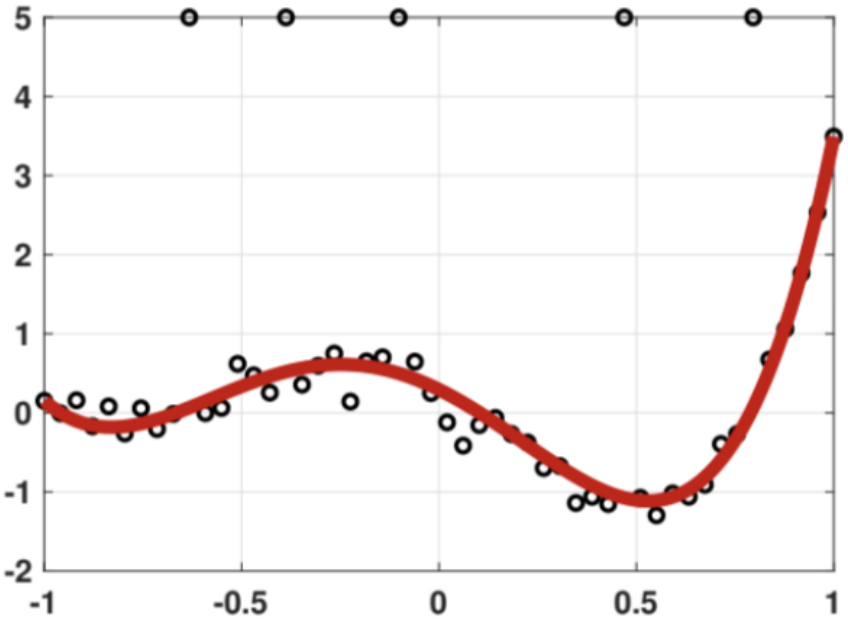
- Data $(X_i, Y_i), i = 1, \ldots, n.$
- To find a network
  $f(x; \theta)$
  such that
  $\sum(Y_i - f(X_i; \theta))^2$
  is minimized over

$\mathcal{F} = \{f: f(x; \theta)$ *is a neural network parameterized by* $\theta \in \mathbb{R}^s\}$

- How do we solve for θ ?
  1. Initialize $\theta_0 \in \mathbb{R}^s$
  2. Calculate the gradient at θ_t
  3. Move a step α_t
  4. Iterate until stop.

Revisit: Deep Nonparametric Regression

Recall the regression problem

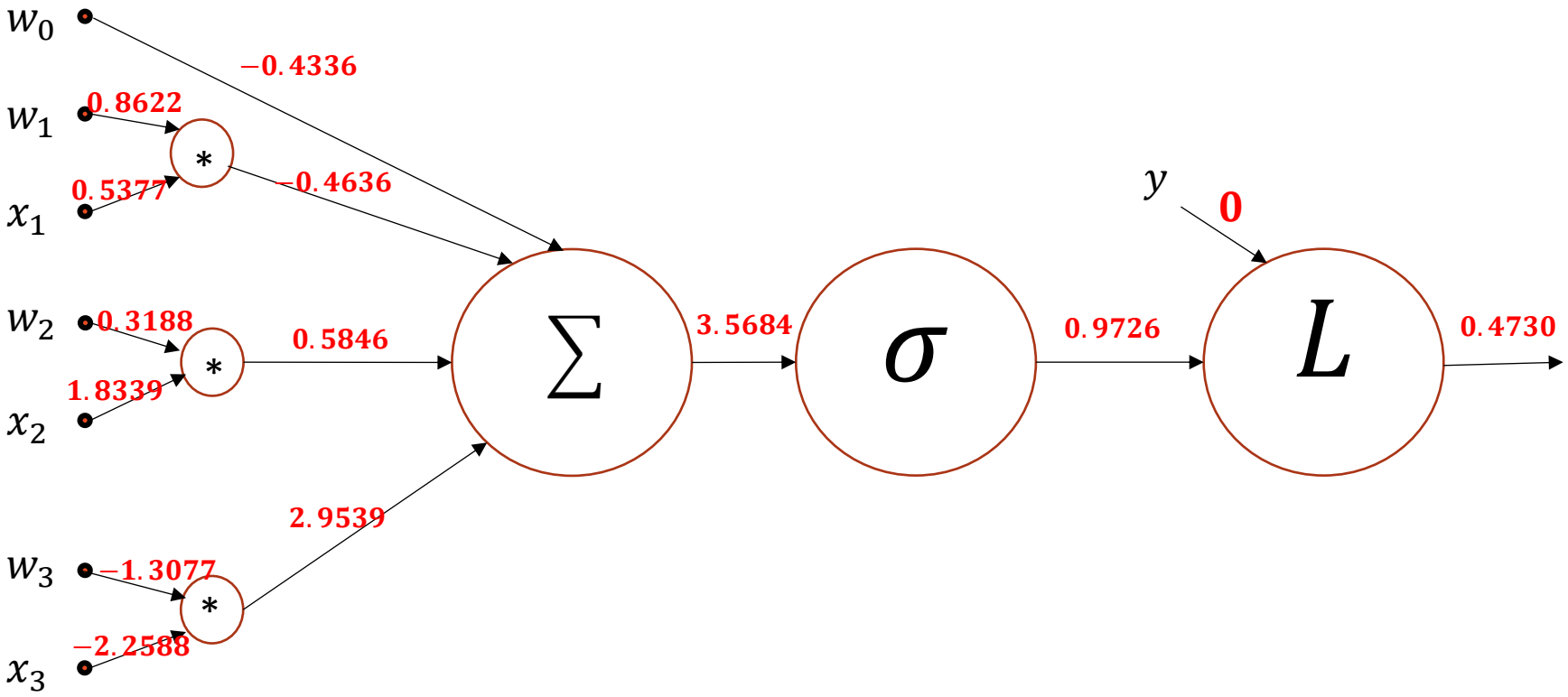


- Data $(X_i, Y_i), i = 1, \ldots, n.$
- To find a network
  $f(x; \theta)$
  such that
  $\sum \phi(Y_i - f(X_i; \theta))$
  is minimized over

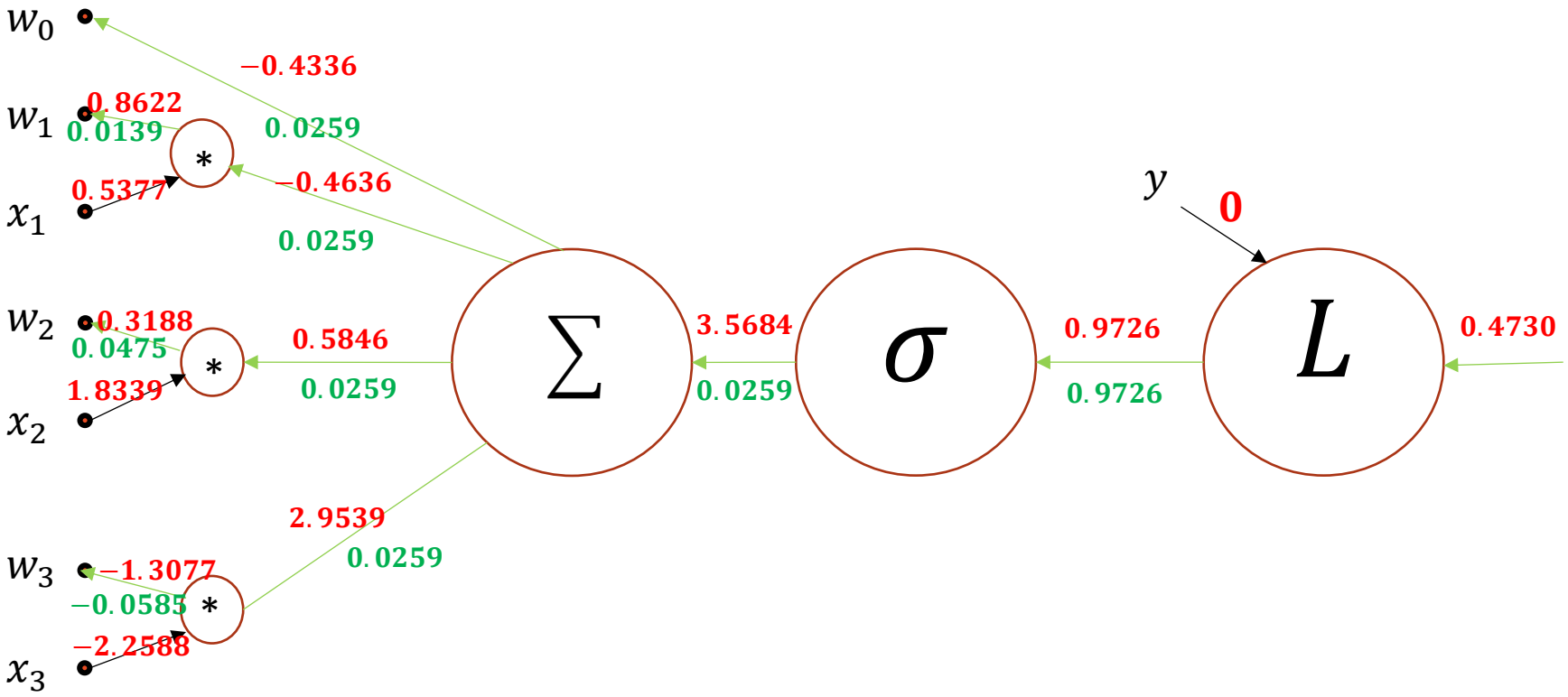
$\mathcal{F} = \{f: f(x; \theta)$ *is a neural network parameterized by* $\theta \in \mathbb{R}^s\}$

- How do we solve for θ ?
  1. Initialize $\theta_0 \in \mathbb{R}^s$
  2. Calculate the gradient at θ_t (with different ϕ)
  3. Move a step α_t
  4. Iterate until stop

Backpropogation



A simple example

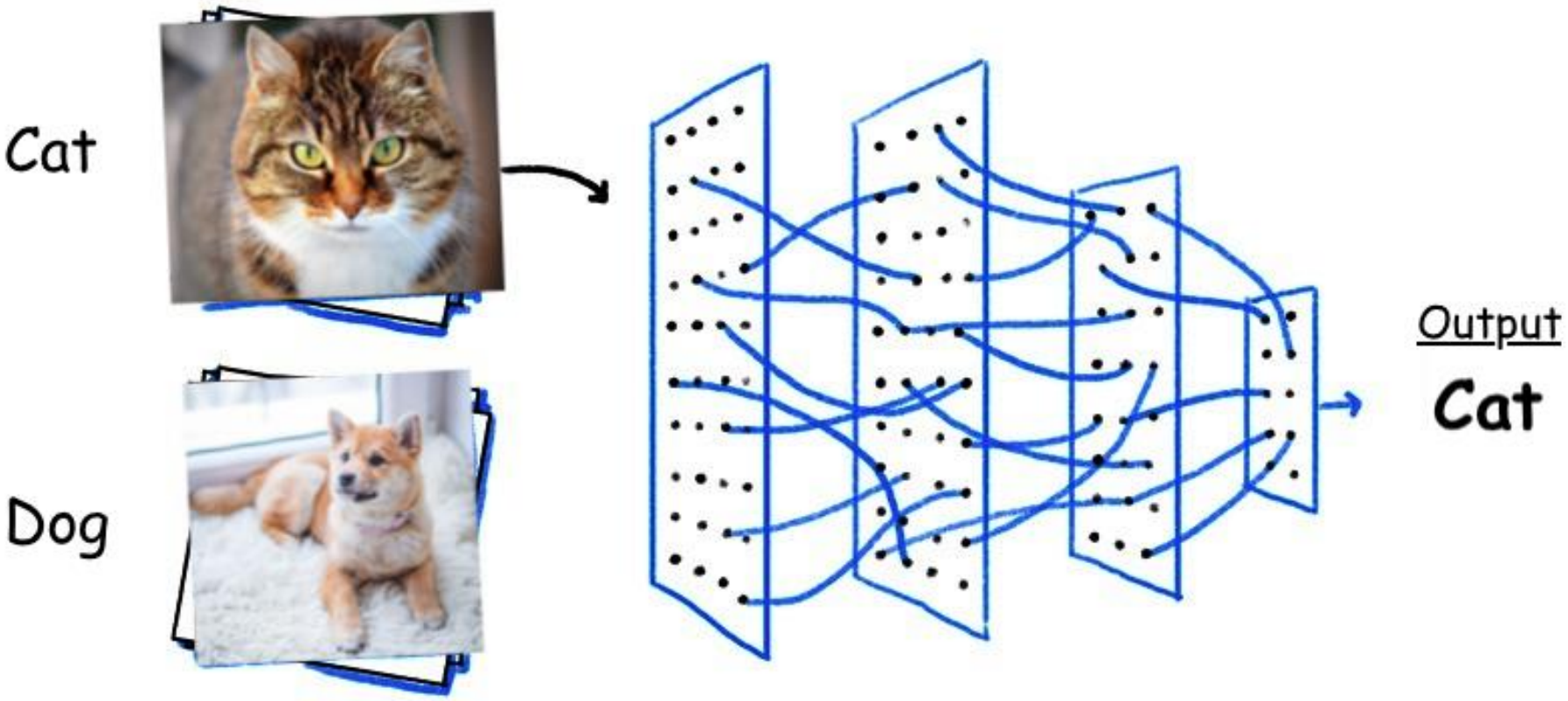


Overview

1. **An Introduction to Computer Vision Problem**
2. **DNN for Image Classification Problems**
3. **Convolution**
4. **Convolutional Neural Network**
5. **Pooling**

Computer Vision Problems

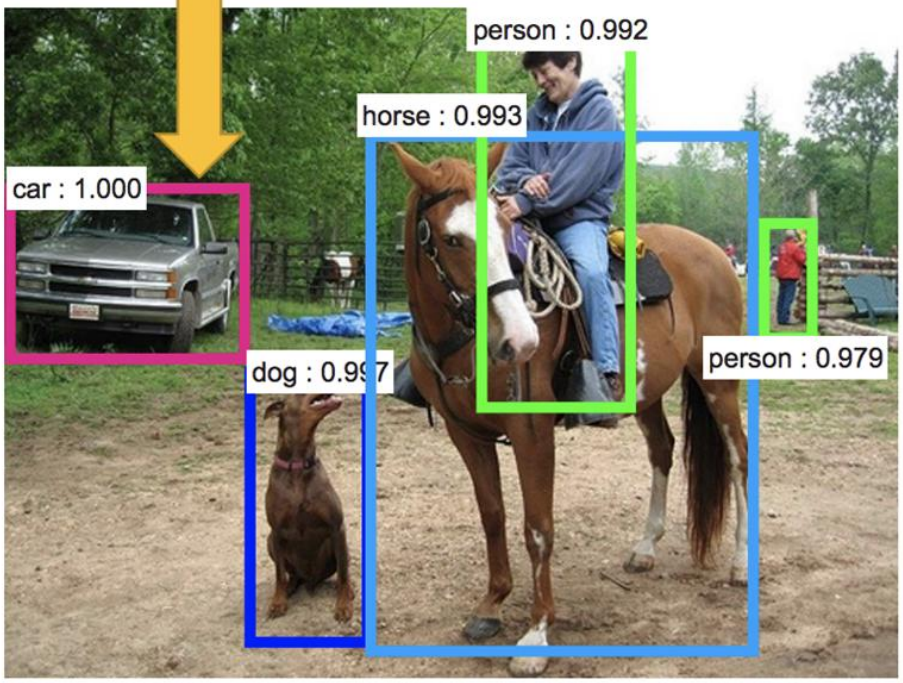
Computer Vision Task: **Image Classification**



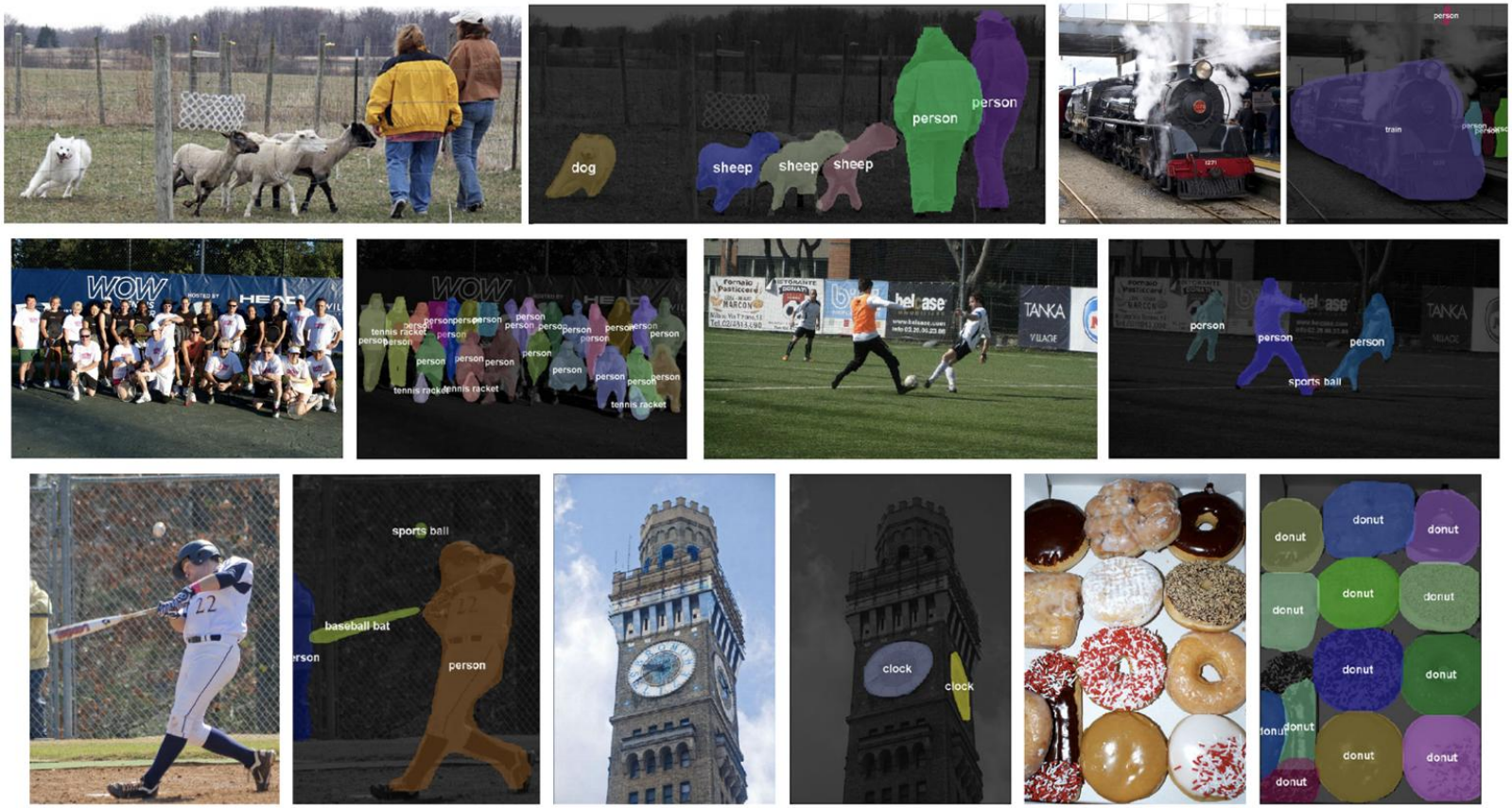
Object Detection = What, and Where

Localization
Where?

Recognition
What?



Computer Vision Task: Object Segmentation



Computer Vision Task: Art Style Transformation

Computer Vision Task: Image Captioning

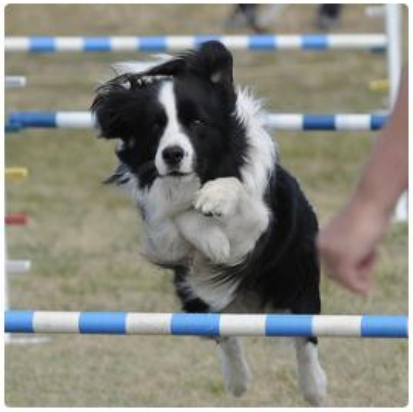
"man in black shirt is playing guitar."

"construction worker in orange safety vest is working on road."

"two young girls are playing with lego toy."

"boy is doing backflip on wakeboard."

"girl in pink dress is jumping in air."



"black and white dog jumps over bar."

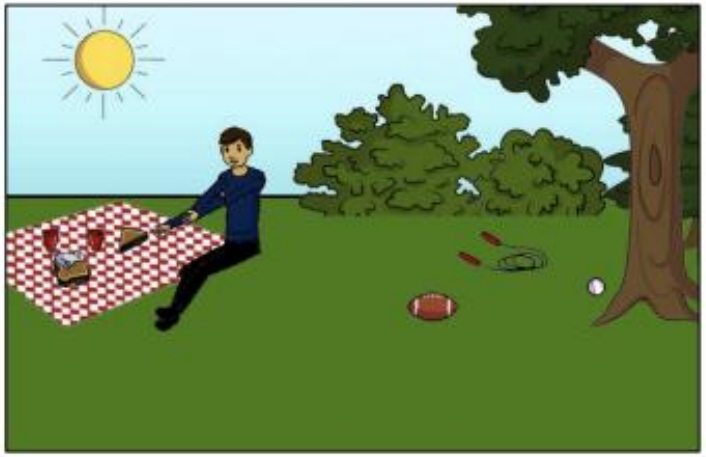
"young girl in pink shirt is swinging on swing."

"man in blue wetsuit is surfing on wave."

Computer Vision Task: Visual Question Answering

What color are her eyes?
What is the mustache made of?

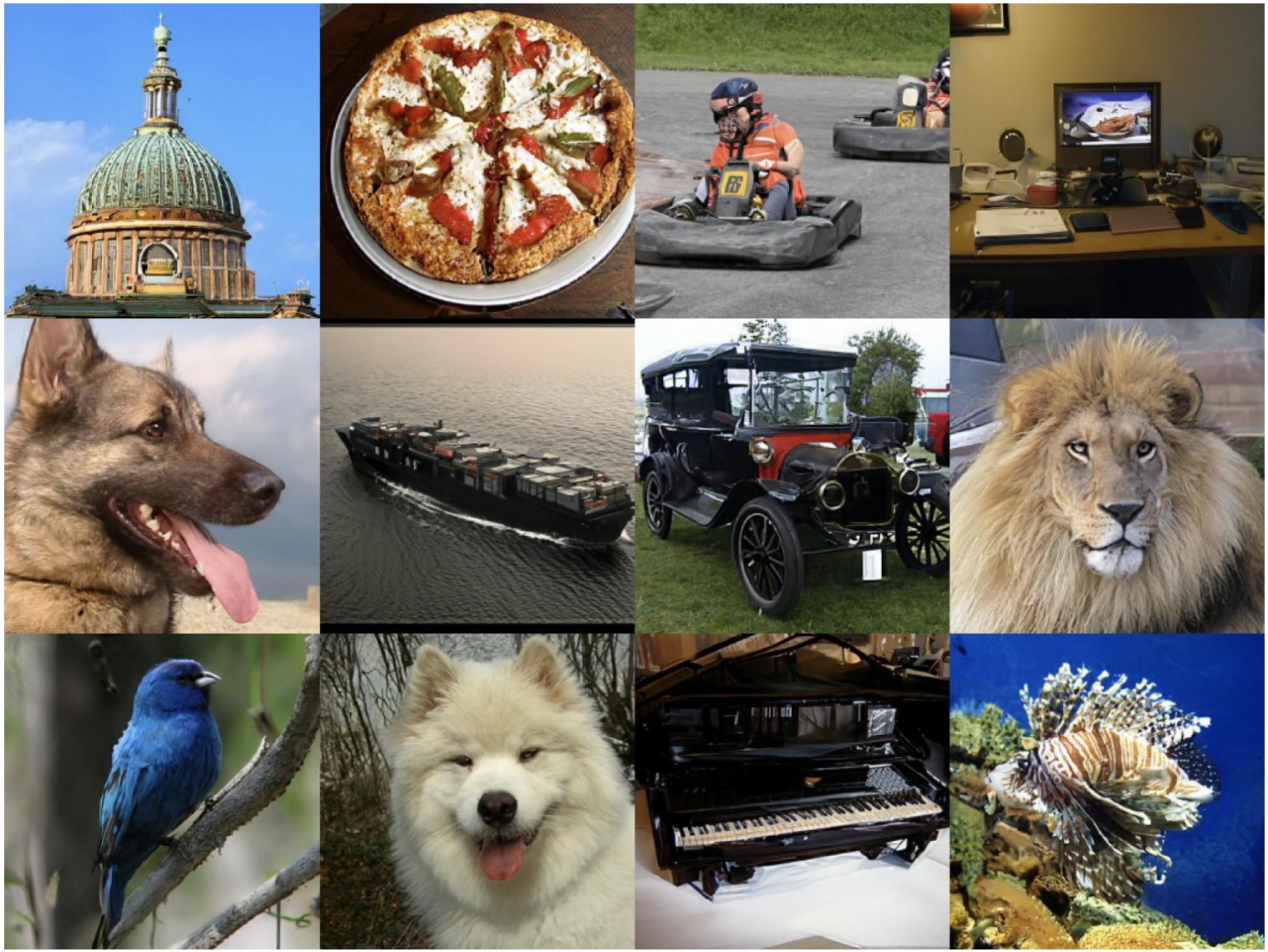
How many slices of pizza are there?
Is this a vegetarian pizza?



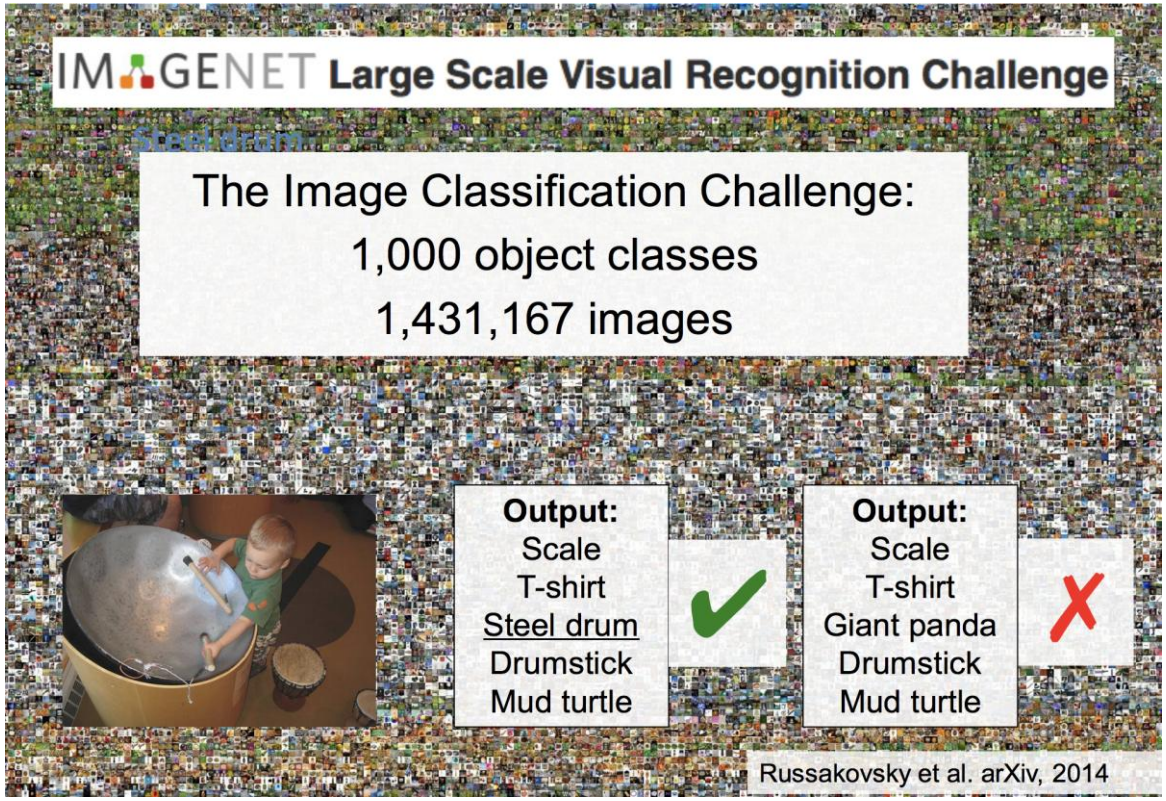
Is this person expecting company?
What is just under the tree?

Does it appear to be rainy?
Does this person have 20/20 vision?

Computer Vision Task: Image Generation



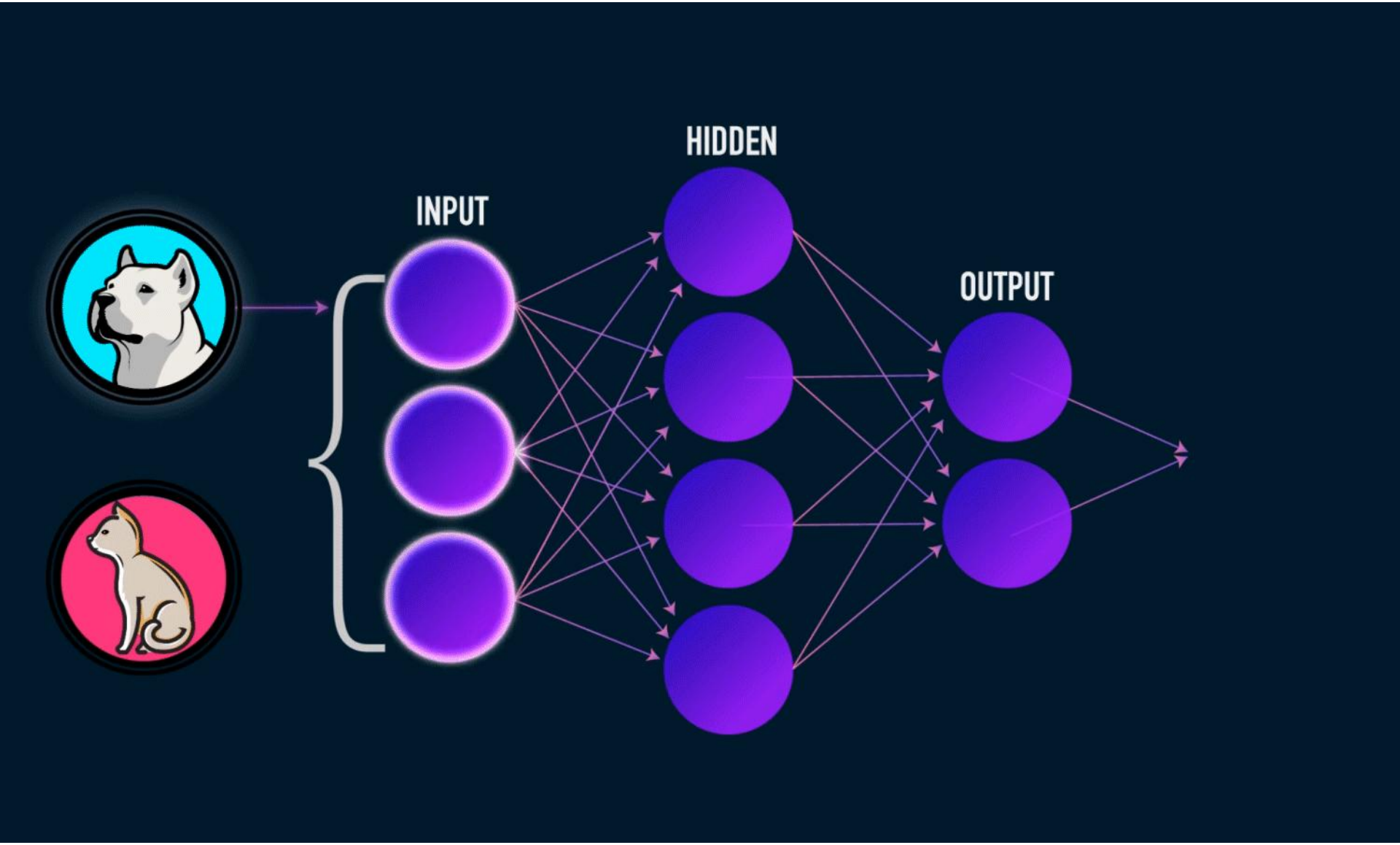
Oil for Computer Vision: IMAGENET



Feifei Li

DNN for Image Classification Problem

Image Classification with DNN



Recap: Fully Connected DNN

Deep Neural Networks (Multi-Layer Perceptrons)

- The architecture of a MLP is expressed as a composition of a series of functions

$$f_\theta(x) = \mathcal{A}_L \circ \sigma \circ \mathcal{A}_{L-1} \circ \sigma \circ \cdots \circ \sigma \circ \mathcal{A}_1(x), \quad x \in \mathbb{R}^{d_0}$$

where

$$\mathcal{A}_i(x) = W_i x + b_i, \qquad x \in \mathbb{R}^{d_{i-1}}$$

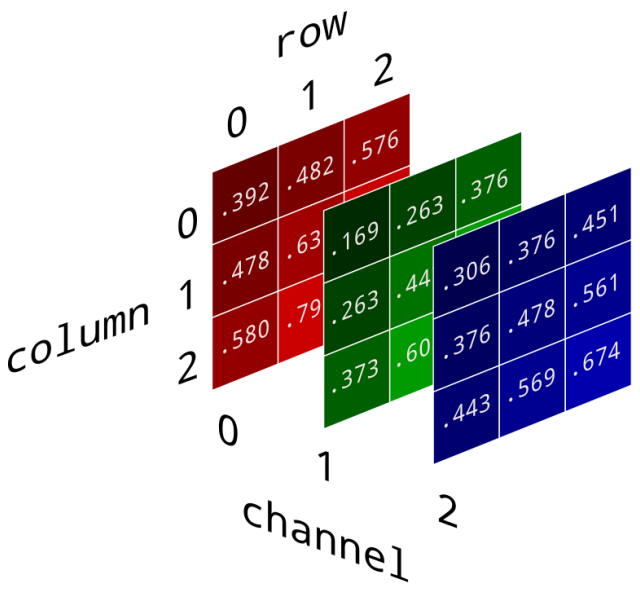
is the i-th linear transformation with

weight matrix $W_i \in \mathbb{R}^{d_i \times d_{i-1}}$ and bias vector $b_i \in \mathbb{R}^{d_i}$,

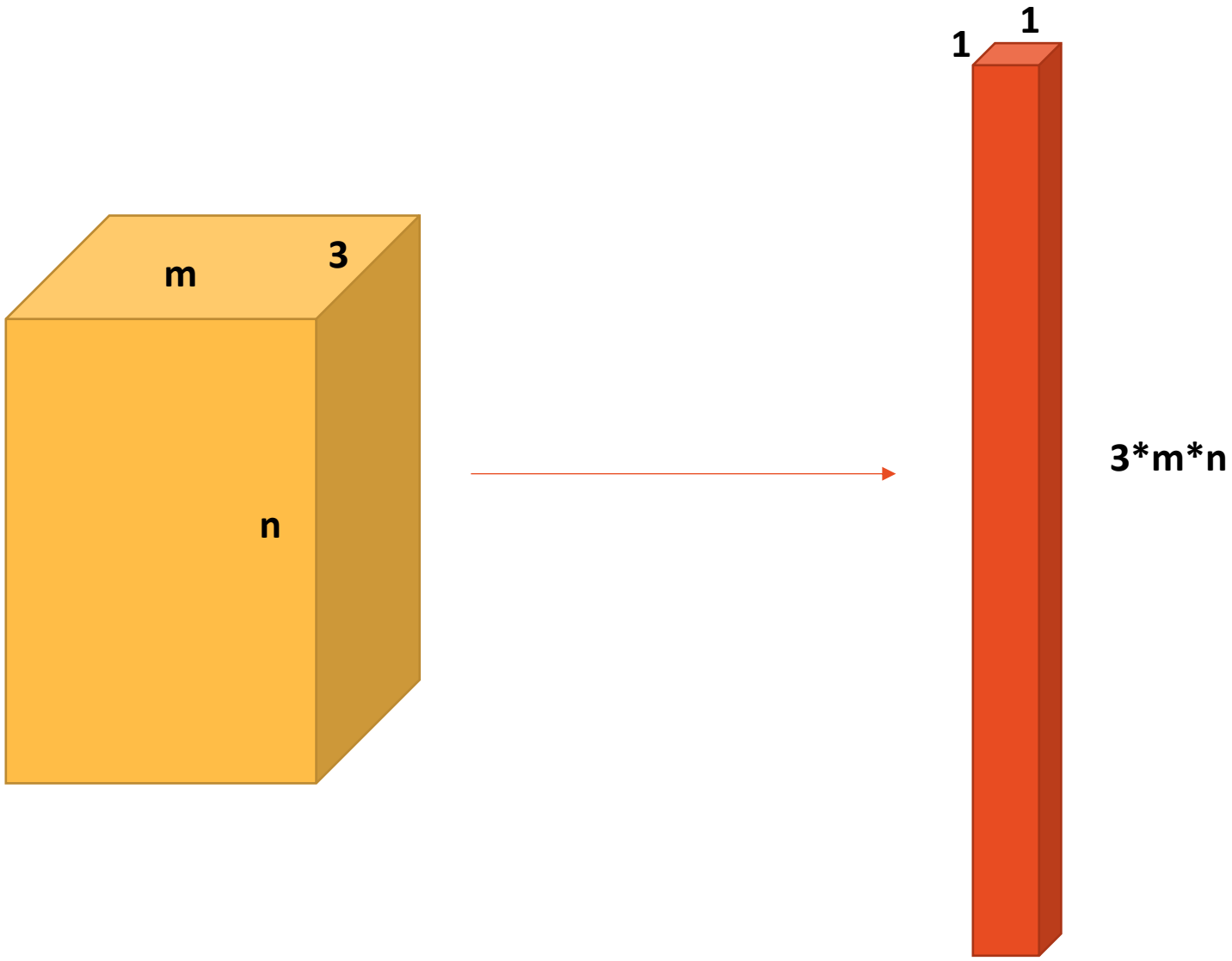
and σ is the activation function,

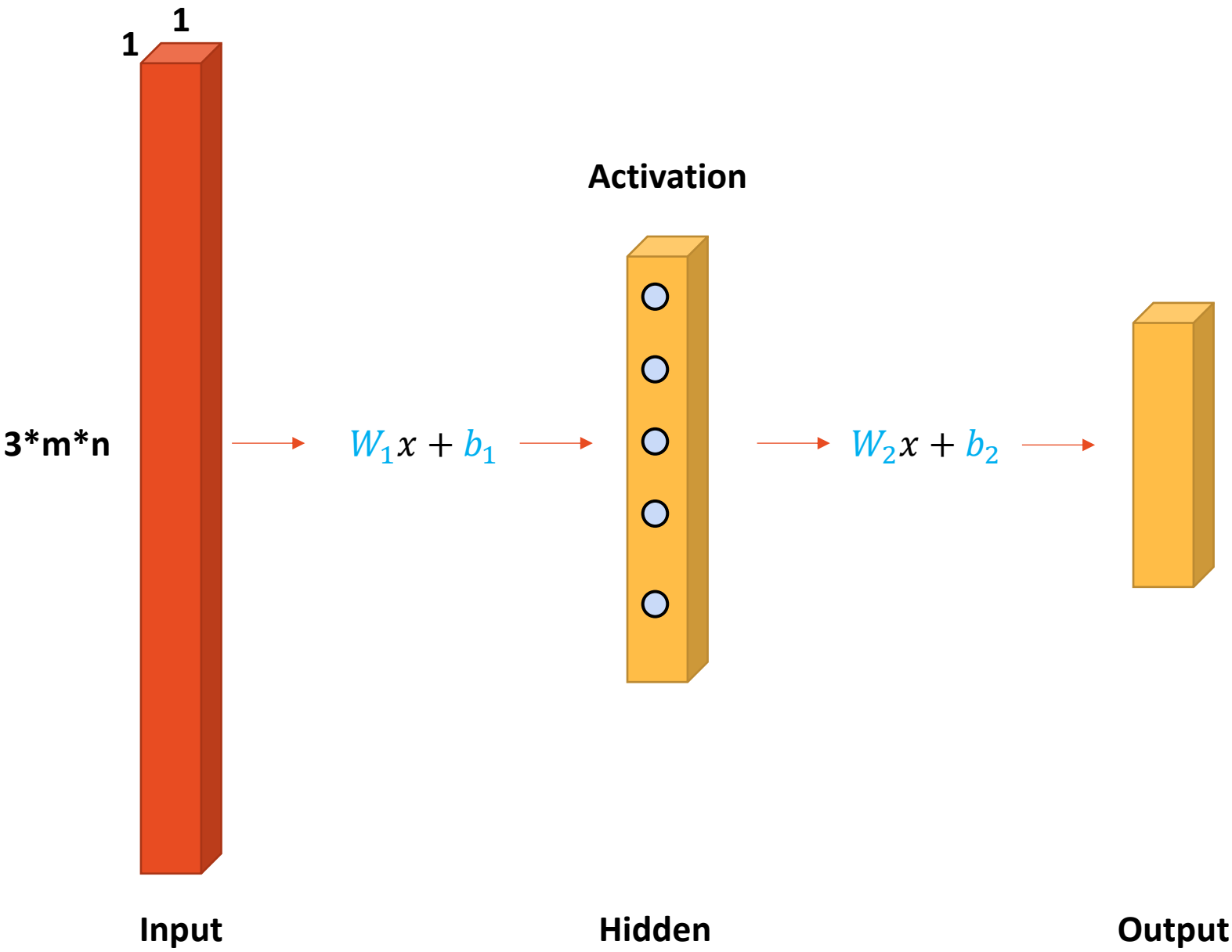
e.g., ReLU $\sigma(x) = \max\{x, 0\}$. Sigmoid $\sigma(x) = (1 + e^{-x})^{-1}$.

Color Image as a Tensor

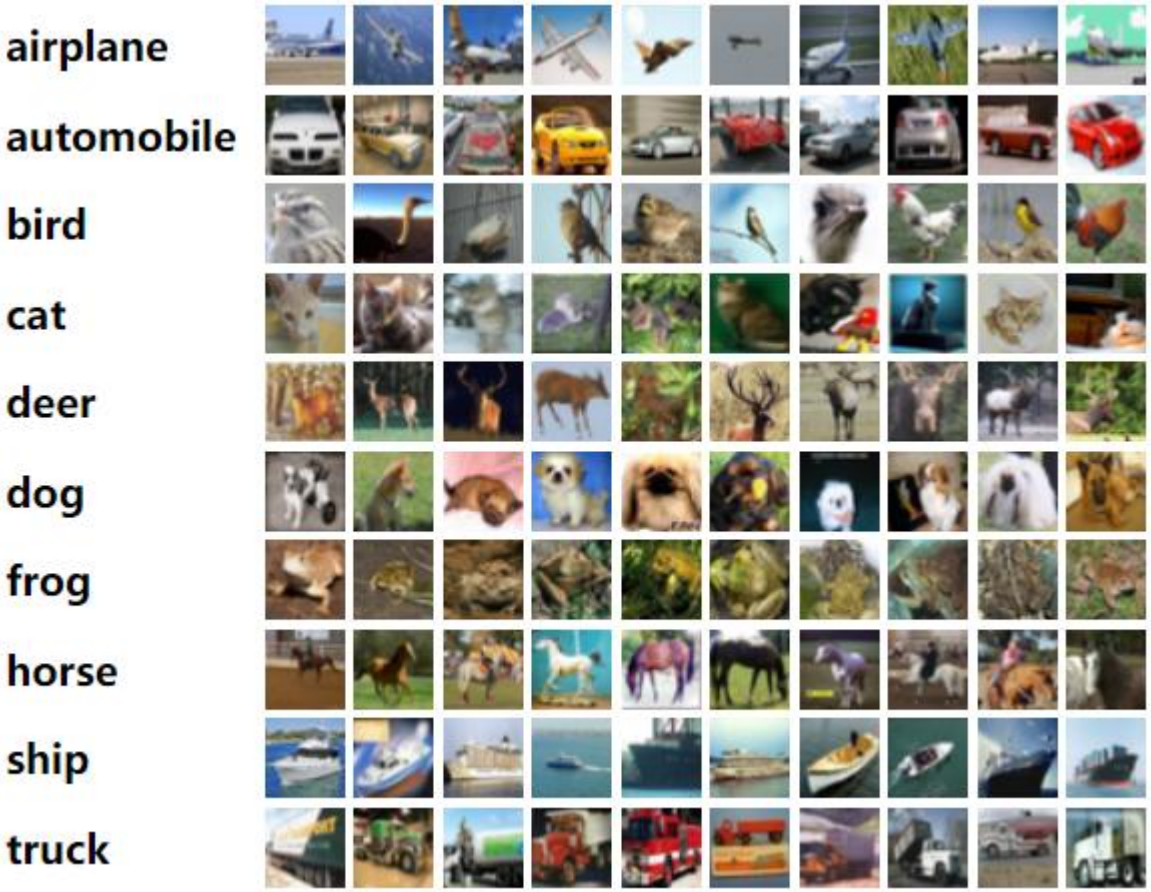


Tensor Reshape

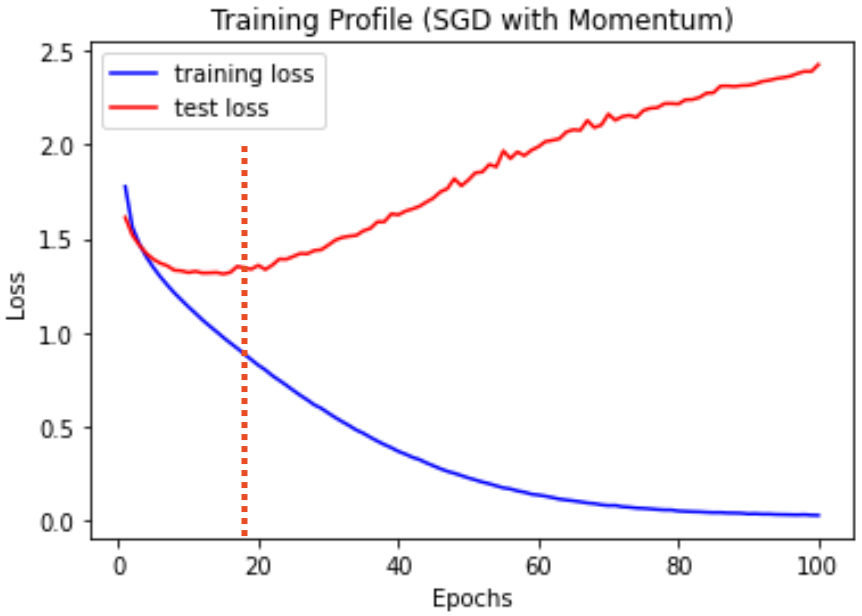



1
1
3*m*n
Input
$W_1x + b_1$
Activation
Hidden
$W_2x + b_2$
Output

A Simple Demonstration on CIFAR 10

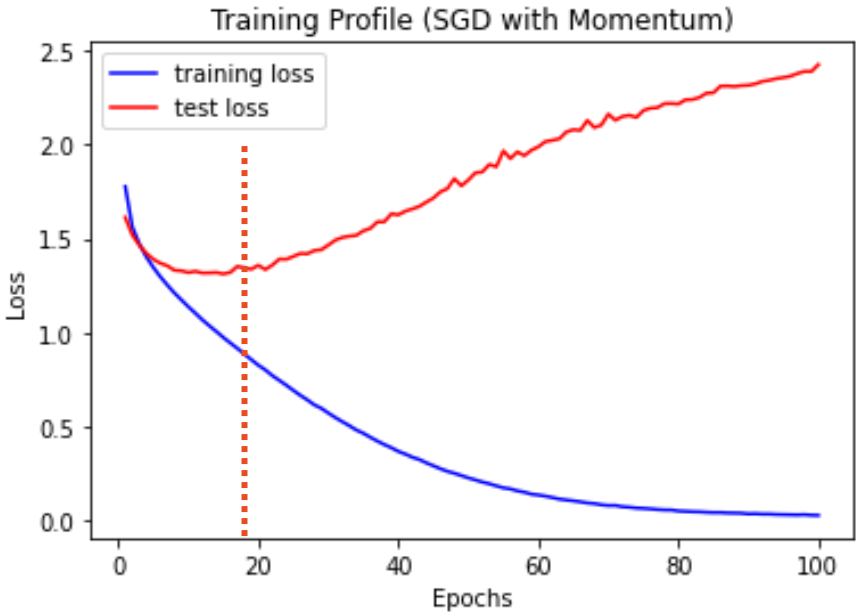


Training Profile for DNN(3204, 512,10)



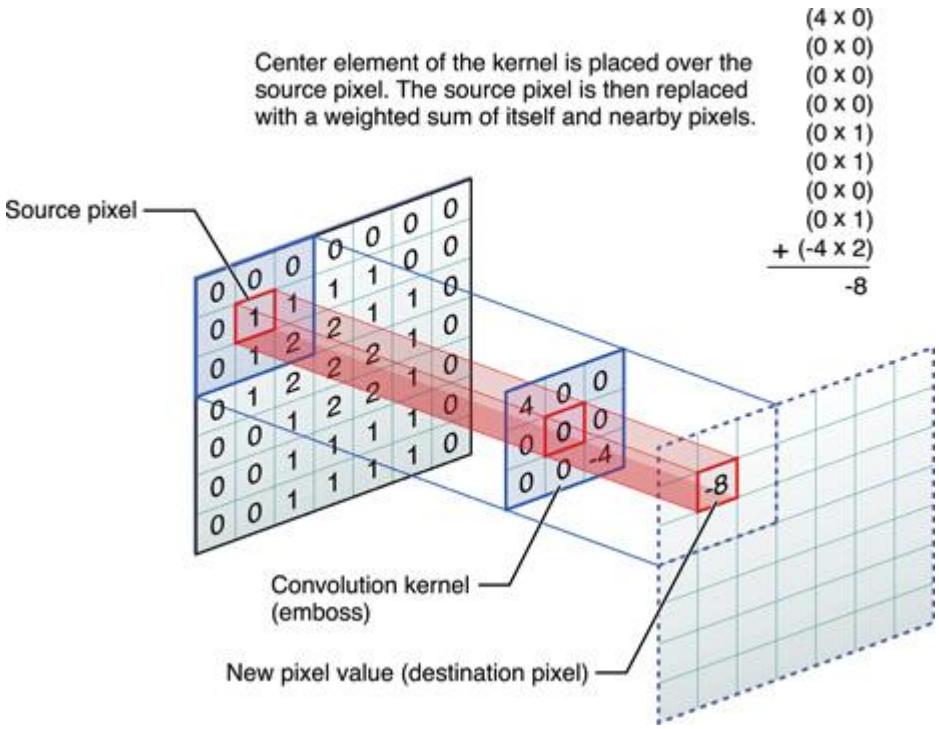
Training Profile for DNN(3204, 512,10)

Overfitting!!!



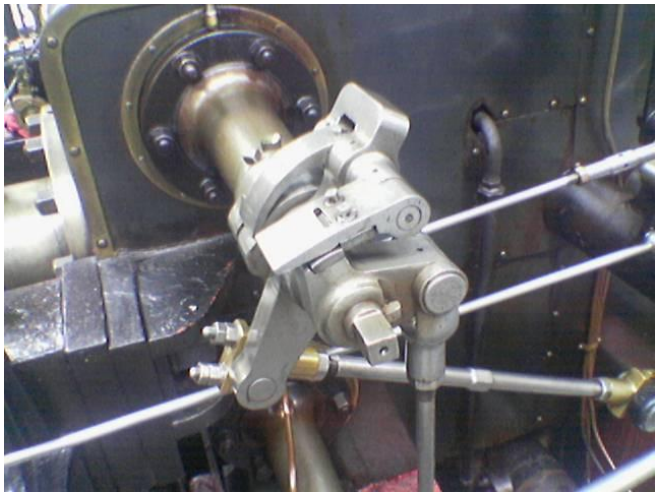
Convolution

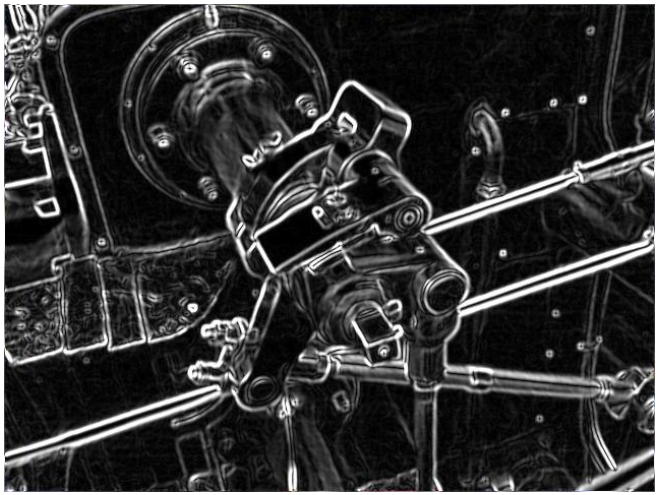
Convolution



Key Advantage: Contain Spatial Information.

Convolution is “NOT” New

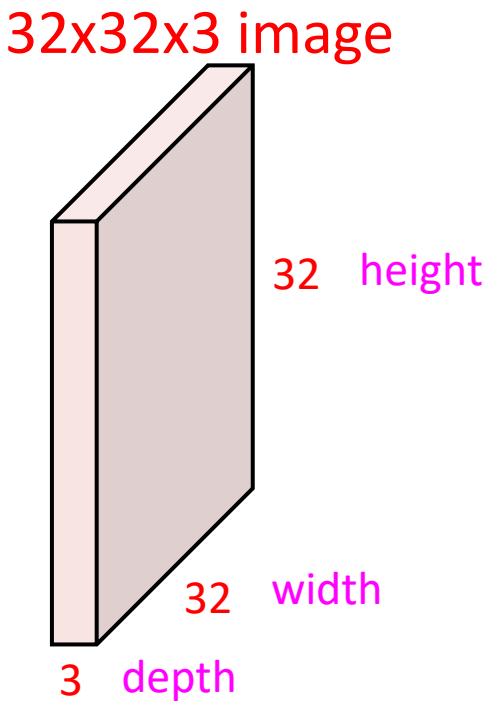




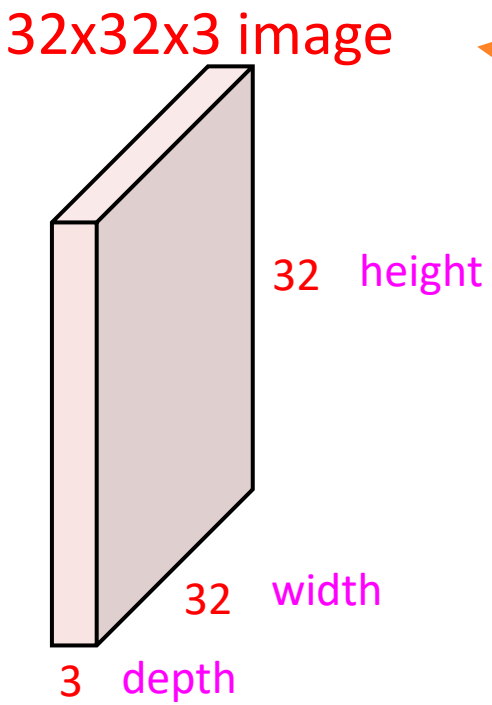
Sobel operator:

$$\mathbf{G}_x = \begin{bmatrix} +1 & 0 & -1 \\ +2 & 0 & -2 \\ +1 & 0 & -1 \end{bmatrix} * \mathbf{A} \quad \text{and} \quad \mathbf{G}_y = \begin{bmatrix} +1 & +2 & +1 \\ 0 & 0 & 0 \\ -1 & -2 & -1 \end{bmatrix} * \mathbf{A}$$

Convolution Layer

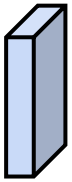


Convolution Layer



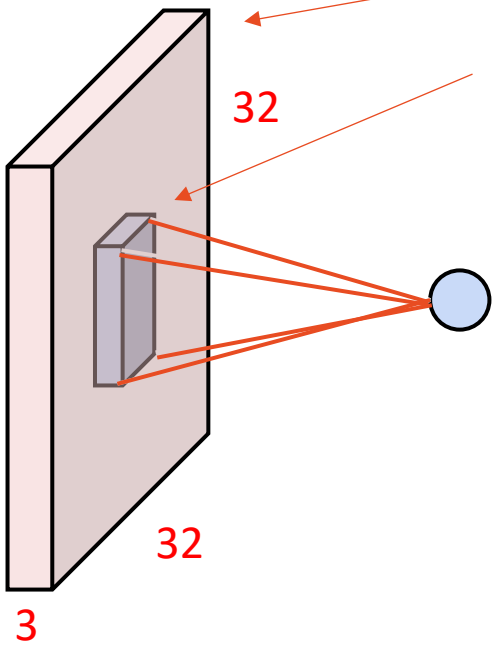
Filters always extend the full depth of the input volume

5x5x3 filter



Convolve the filter with the image i.e. “slide over the image spatially, computing dot products”

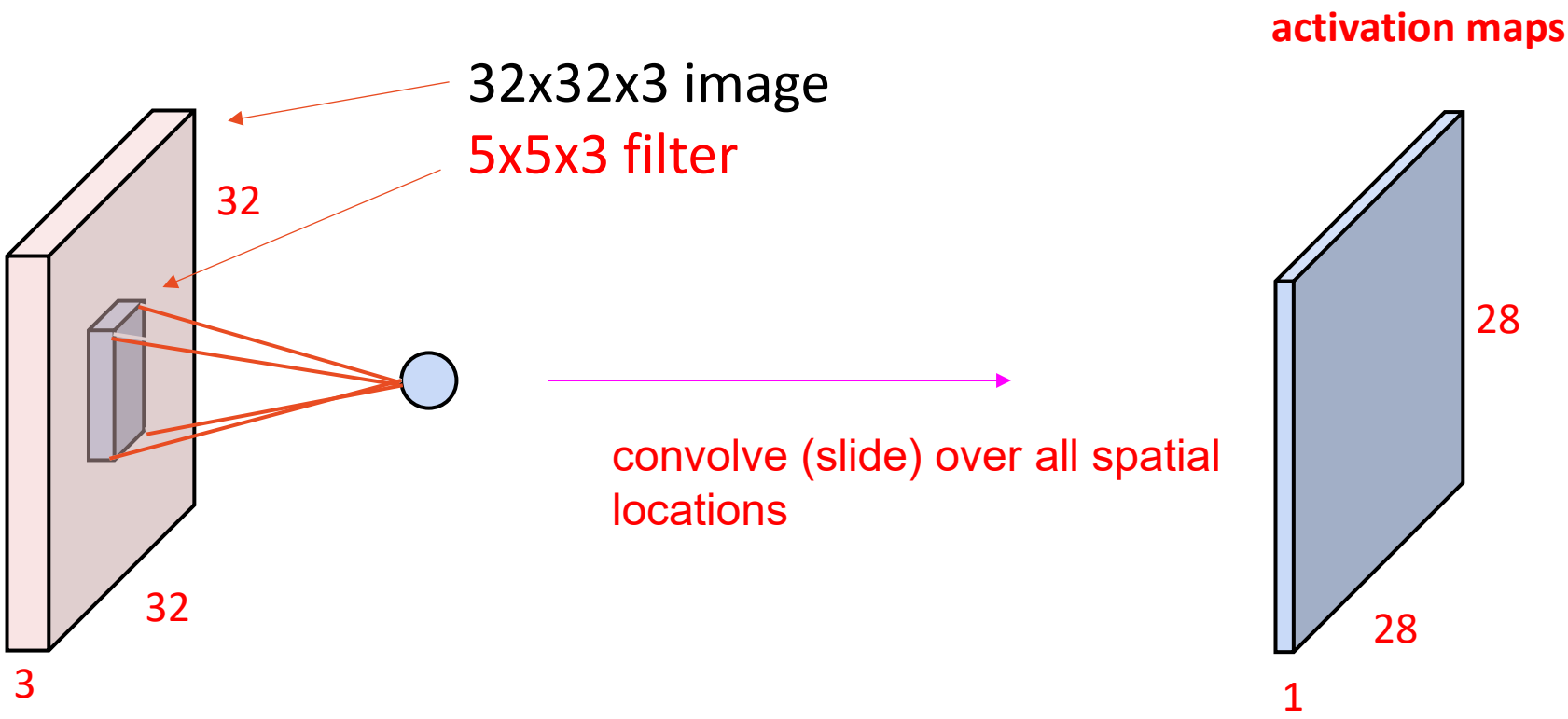
Convolution Layer



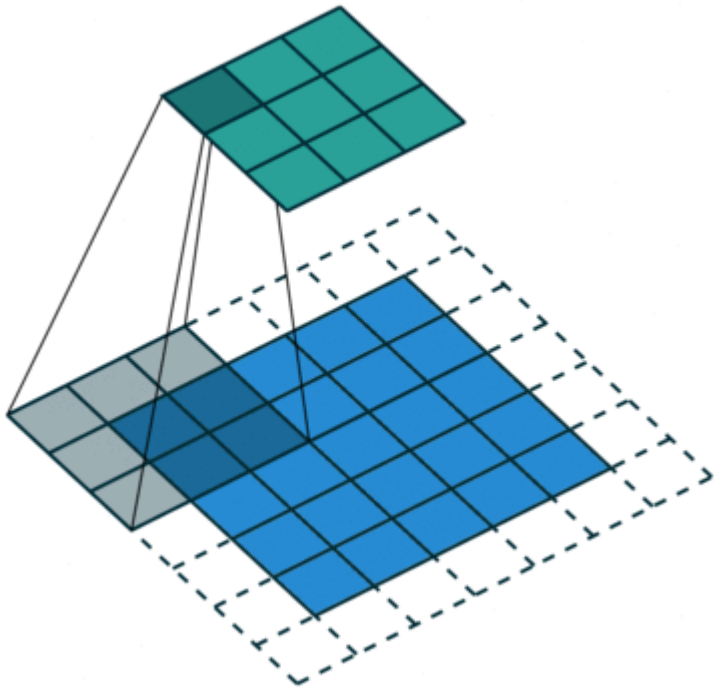
1 number:
the result of taking a dot product between the filter and a small 5x5x3 chunk of the image
(i.e. 5*5*3 = 75-dimensional dot product + bias)

$$w^T x + b$$

Convolution Layer



A Visualization



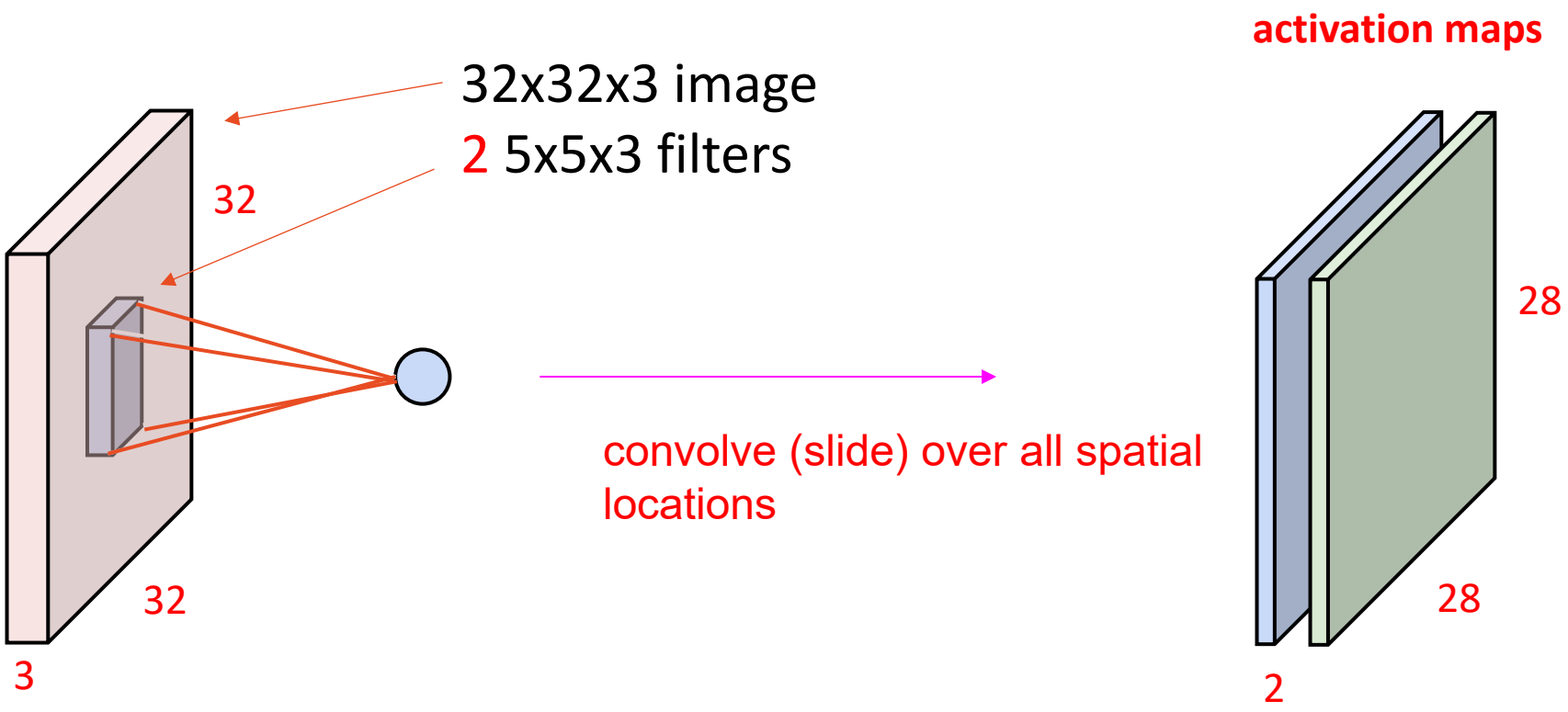
Output

Filter

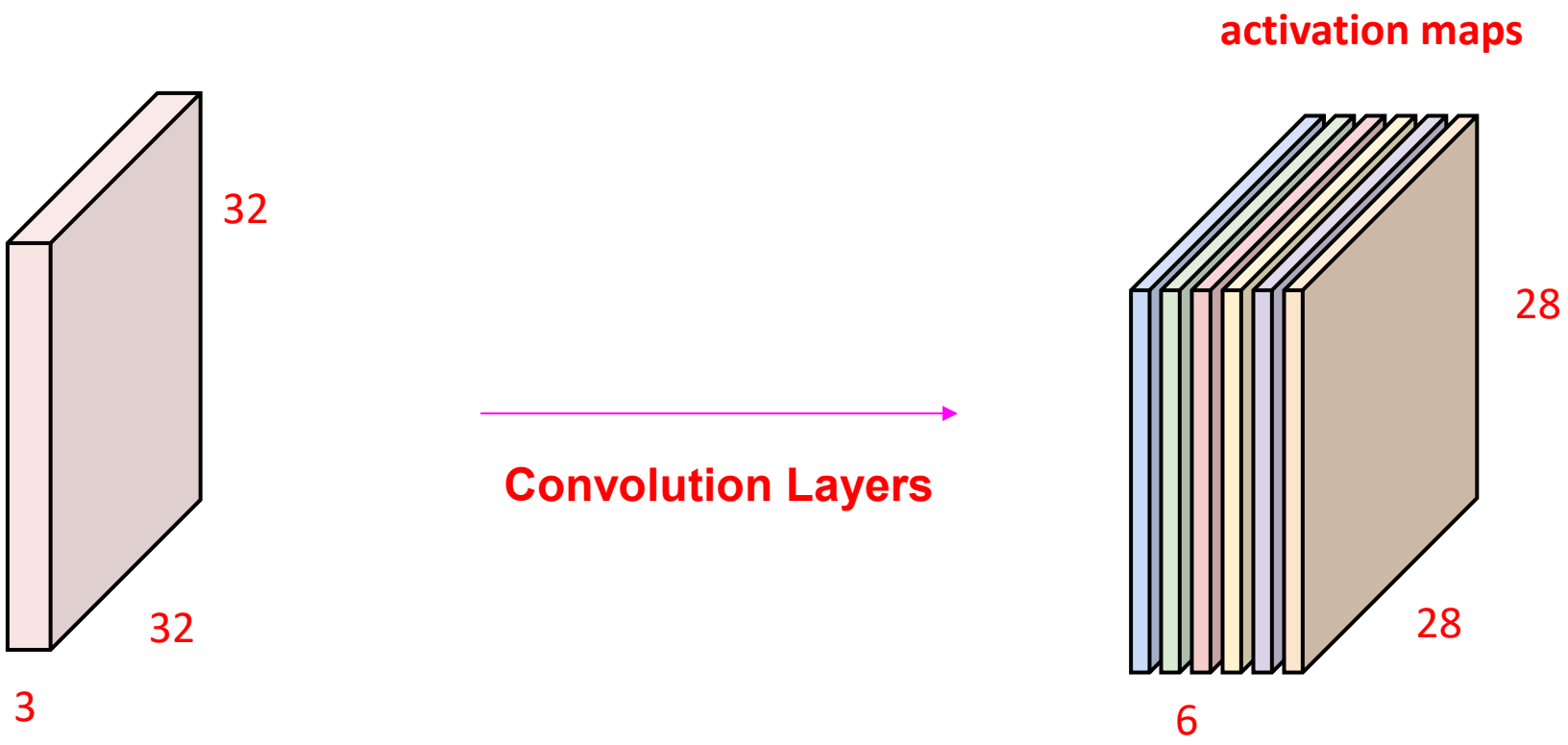
Input

Padding

Convolution Layer

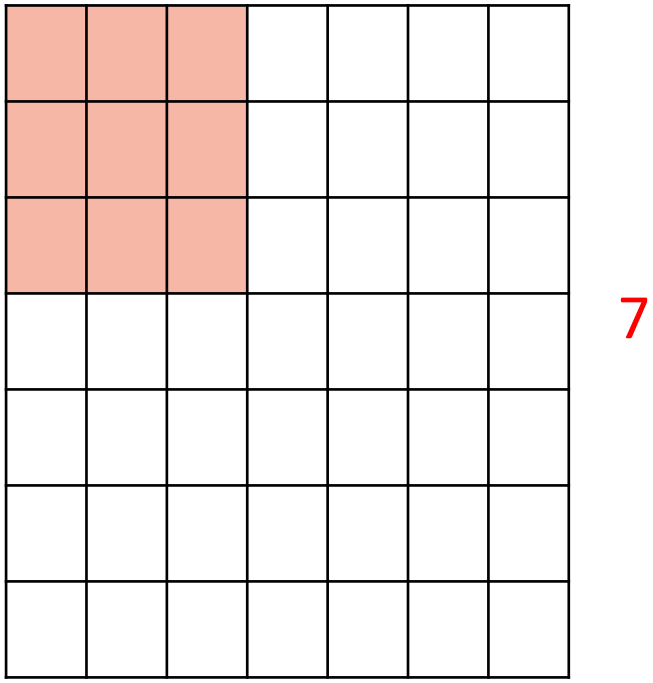


Convolution Layer



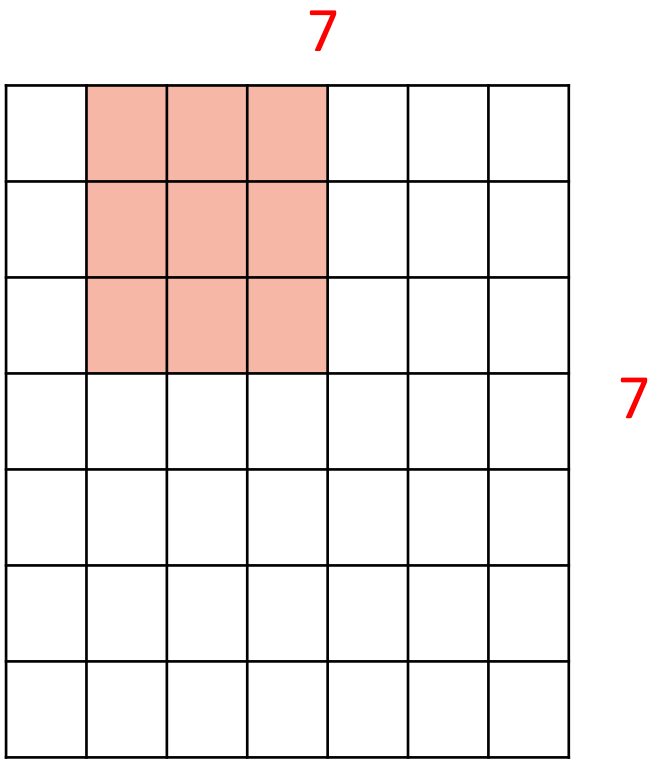
Stack these up to get a new “image” of size 28x28x6!

A Closer Look at Spatial Dimension



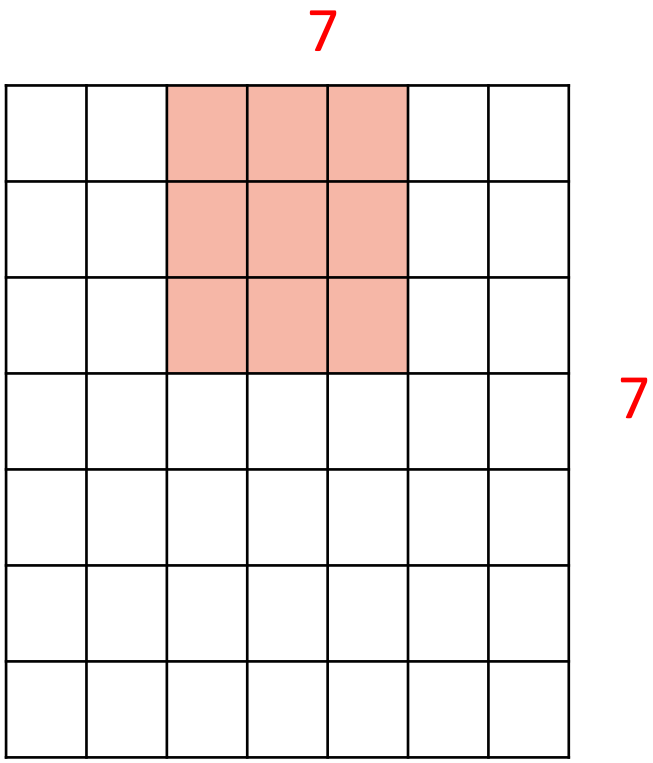
7x7 input (spatially)
assume 3x3 filter

A Closer Look at Spatial Dimension



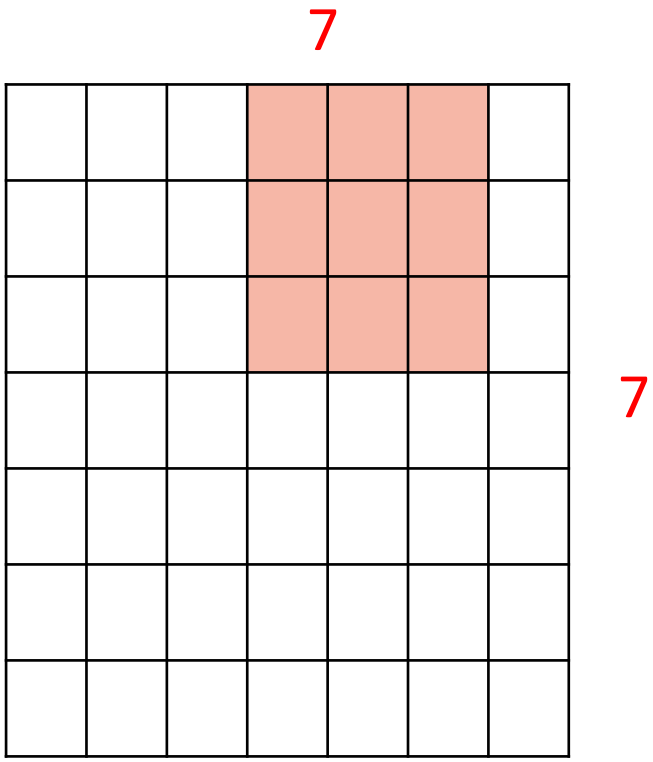
7x7 input (spatially)
assume 3x3 filter

A Closer Look at Spatial Dimension



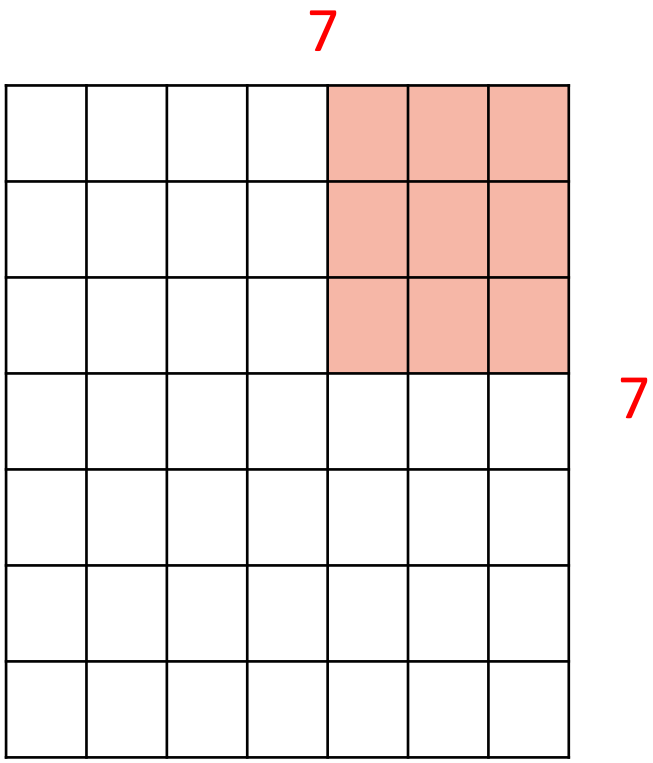
7x7 input (spatially)
assume 3x3 filter

A Closer Look at Spatial Dimension



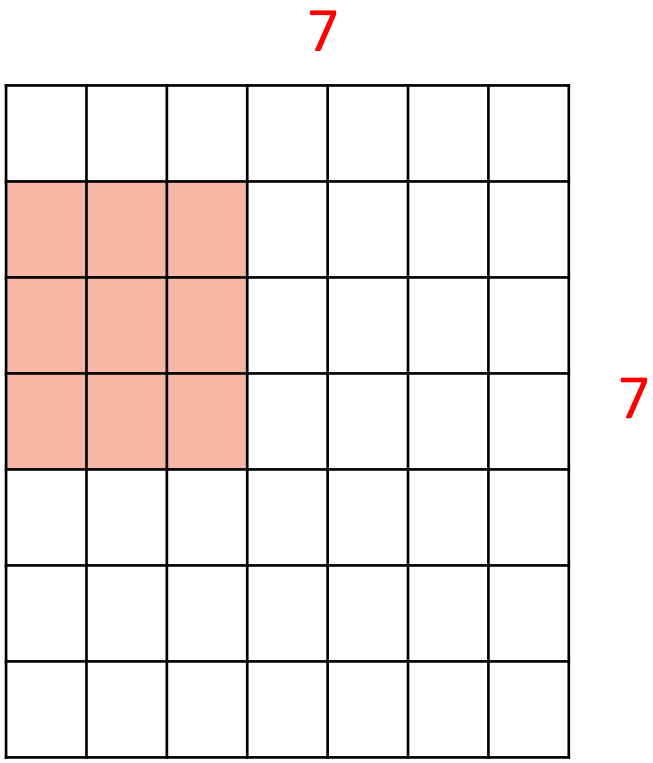
7x7 input (spatially)
assume 3x3 filter

A Closer Look at Spatial Dimension



7x7 input (spatially)
assume 3x3 filter

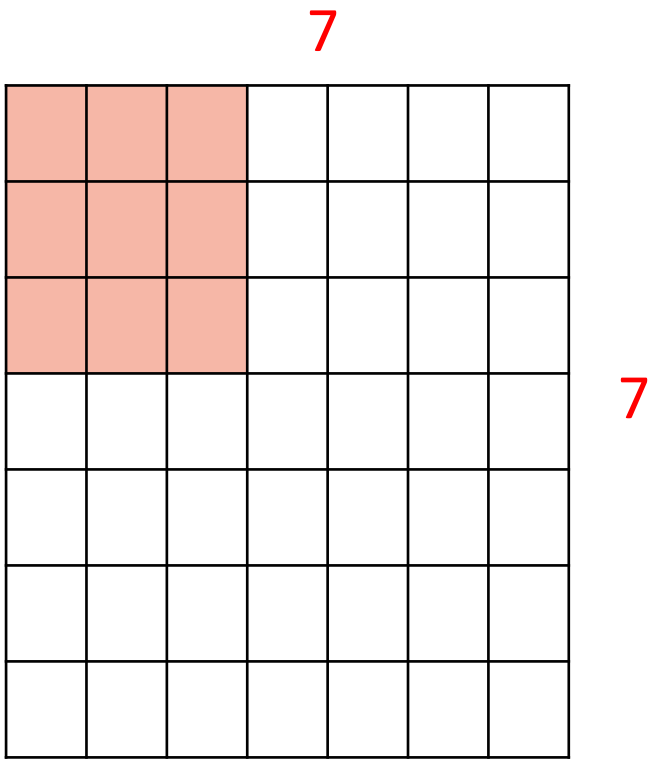
A Closer Look at Spatial Dimension



7x7 input (spatially)
assume 3x3 filter

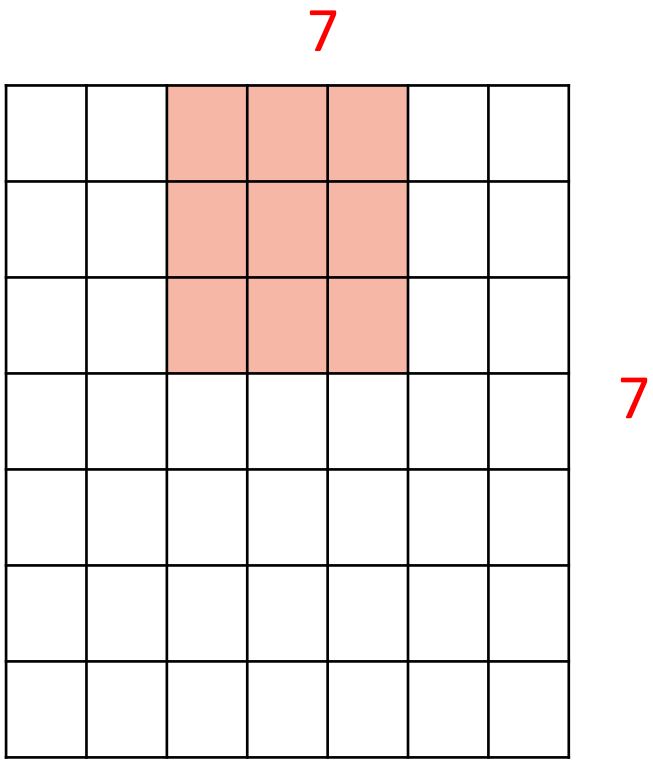
=> 5 x 5 Output

A Closer Look at Spatial Dimension



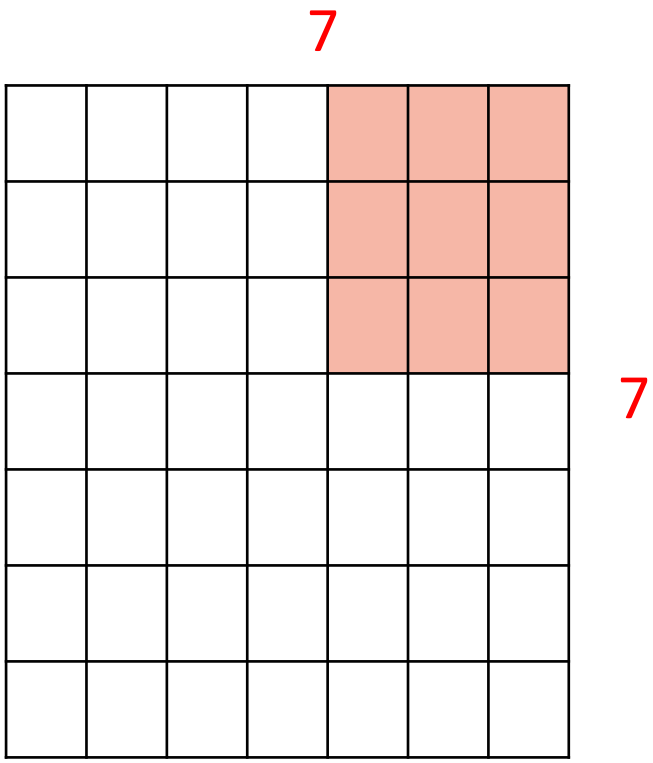
7x7 input (spatially)
assume 3x3 filter
applied with stride 2

A Closer Look at Spatial Dimension



7x7 input (spatially)
assume 3x3 filter
applied with stride 2

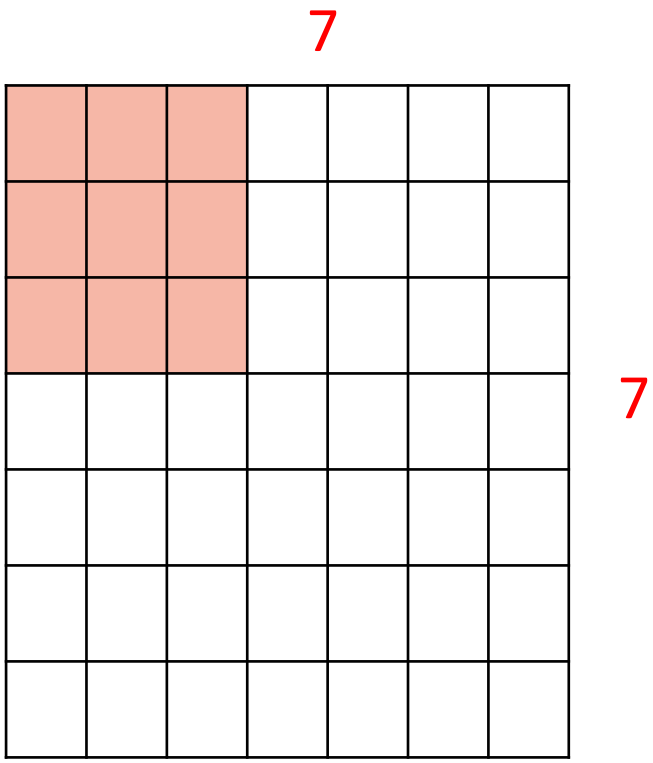
A Closer Look at Spatial Dimension



7x7 input (spatially)
assume 3x3 filter
applied with stride 2

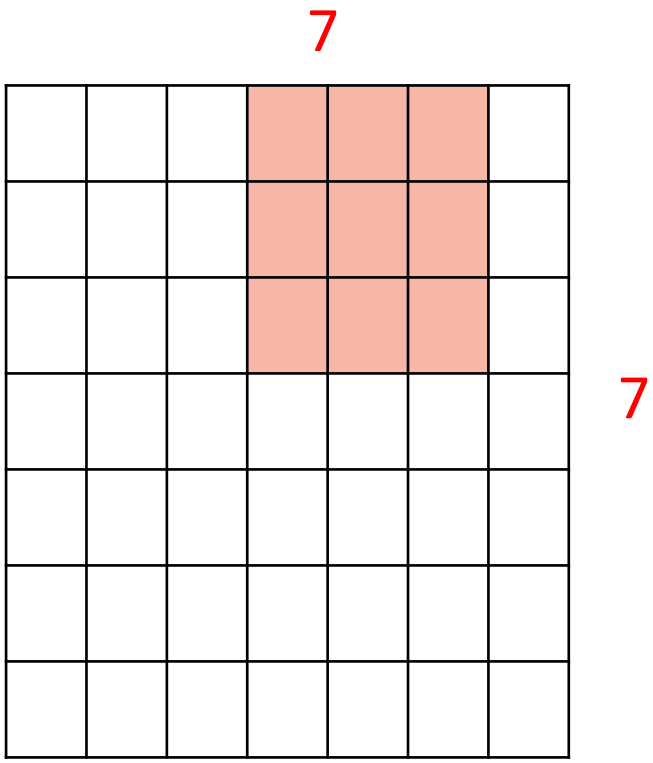
=> 3 x 3 Output

A Closer Look at Spatial Dimension



7x7 input (spatially)
assume 3x3 filter
applied with stride 3 ?

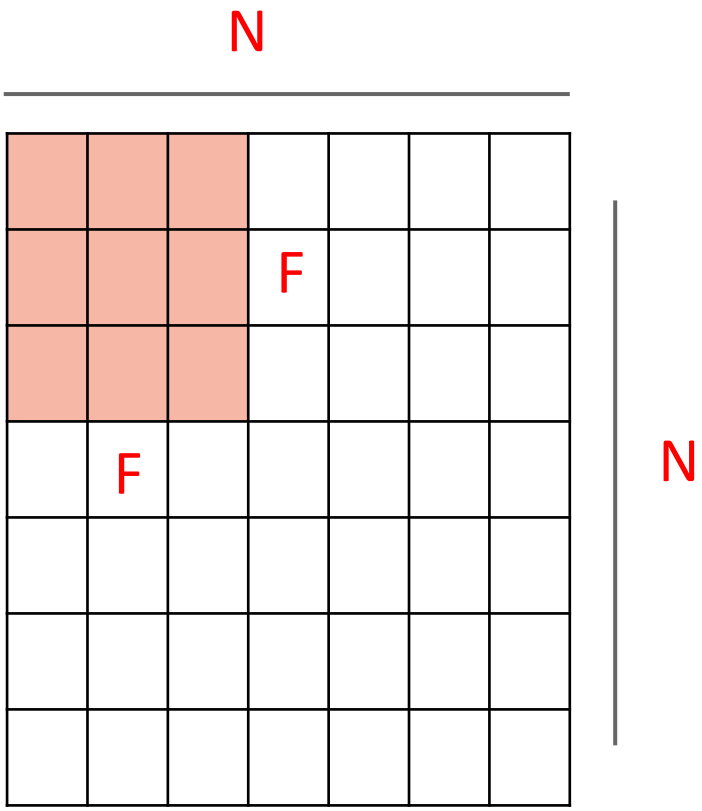
A Closer Look at Spatial Dimension



7x7 input (spatially)
assume 3x3 filter
applied with stride 3 ?

doesn't fit!
cannot apply 3x3 filter on 7x7 input with stride 3.

A Closer Look at Spatial Dimension



Output size:

(N - F) / stride + 1

e.g. N = 7, F = 3:

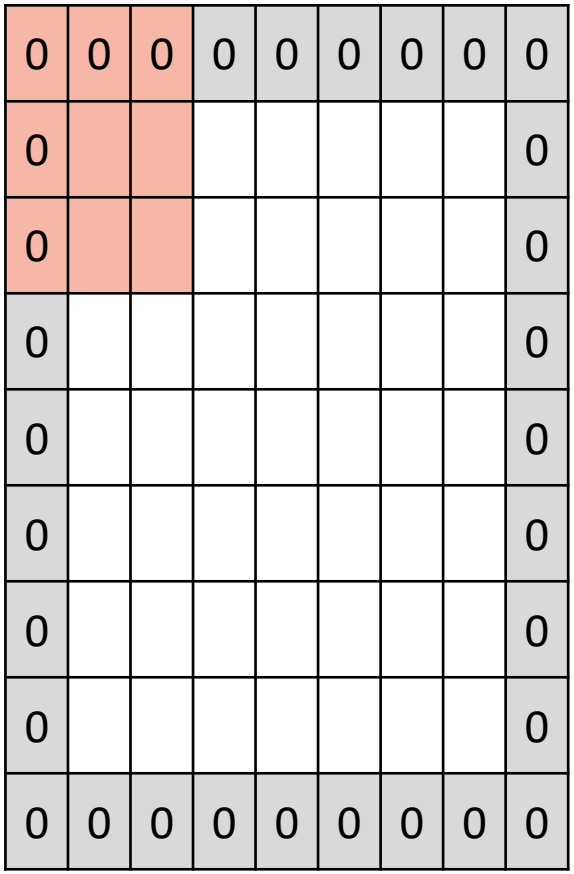
stride 1 => (7 - 3)/1 + 1 = 5

stride 2 => (7 - 3)/2 + 1 = 3

stride 3 => (7 - 3)/3 + 1 = 2.33

How to Handle Dimension Inconsistency ?

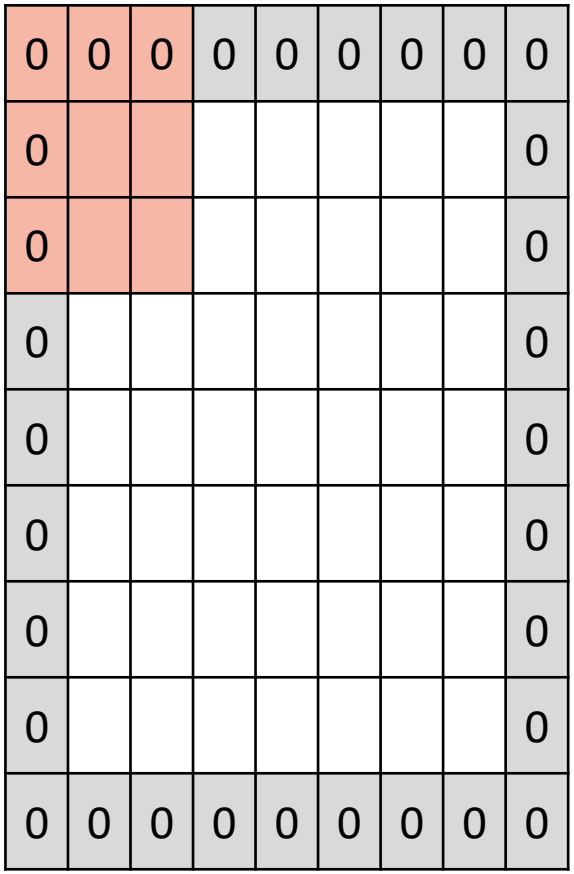
Padding: Pad zeros on the boundary.



e.g. input 7x7
3x3 filter, applied with **stride 1**
pad with 1 pixel border => what is the output?

How to Handle Dimension Inconsistency ?

Padding: Pad zeros on the boundary.



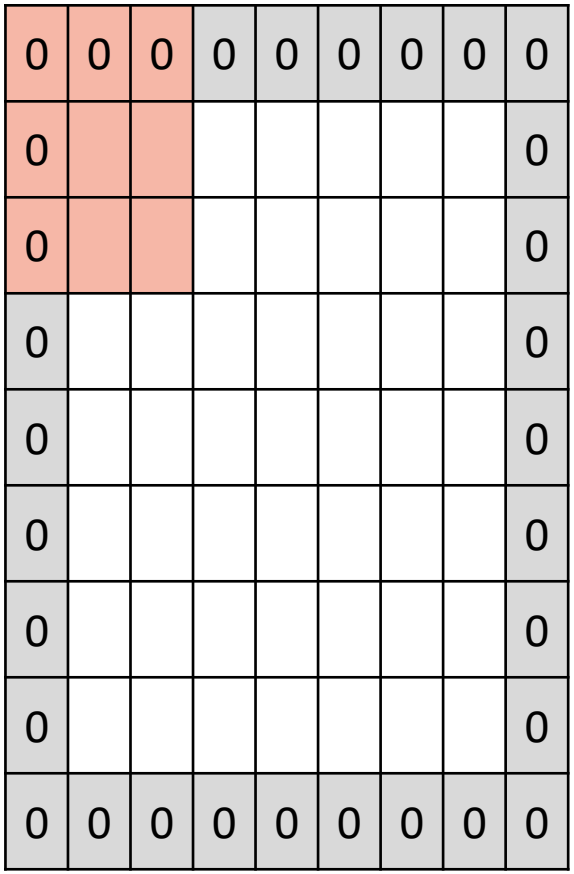
e.g. input 7x7
3x3 filter, applied with **stride 1**
pad with 1 pixel border => what is the output?

7 x 7 Output !

Recall:
(N - F) / stride + 1

How to Handle Dimension Inconsistency ?

Padding: Pad zeros on the boundary.



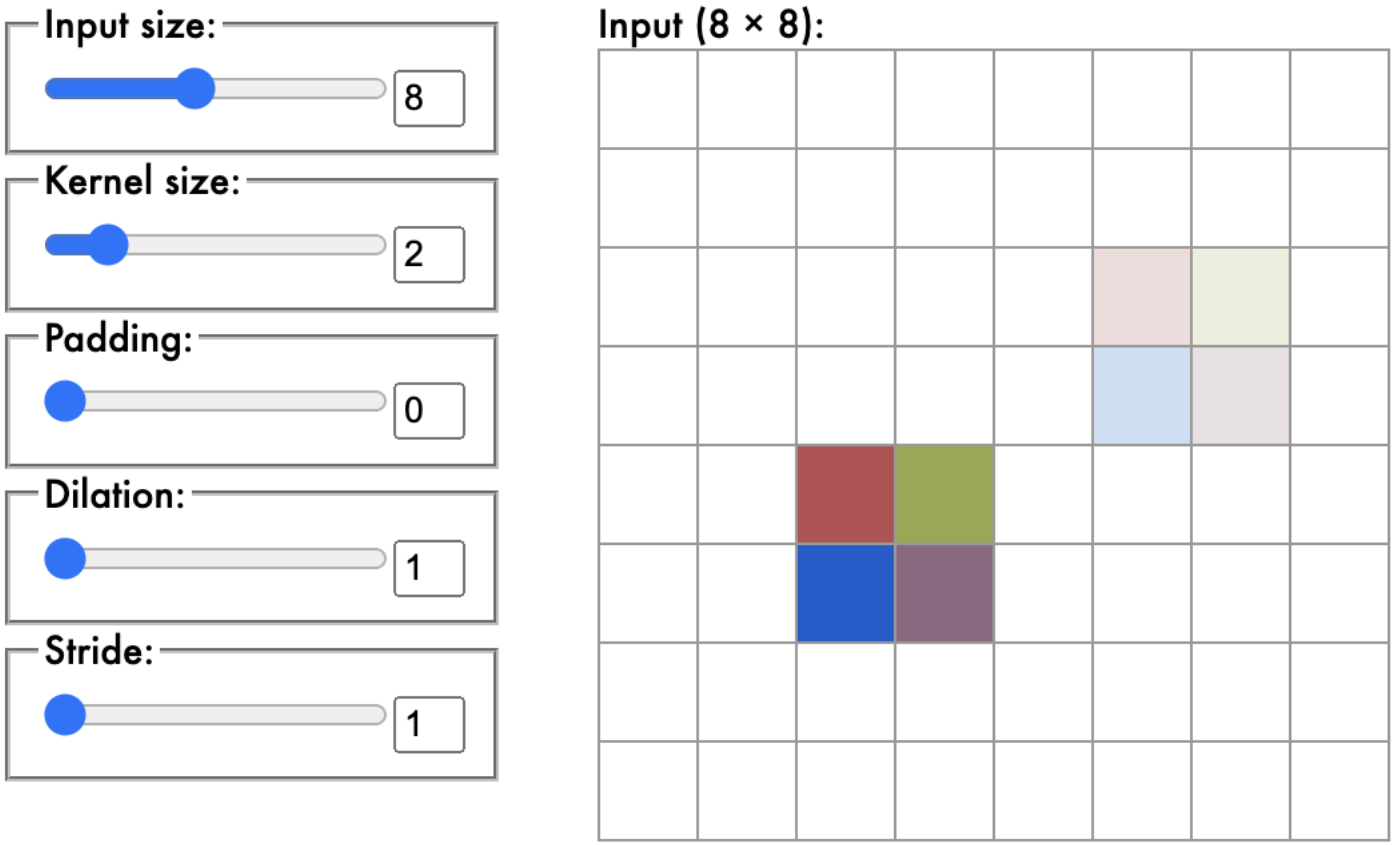
In general, common to see CONV layers with stride 1, filters of size FxF, and zero-padding with (F-1)/2. (will preserve size spatially)

e.g. F = 3 => zero pad with 1

F = 5 => zero pad with 2

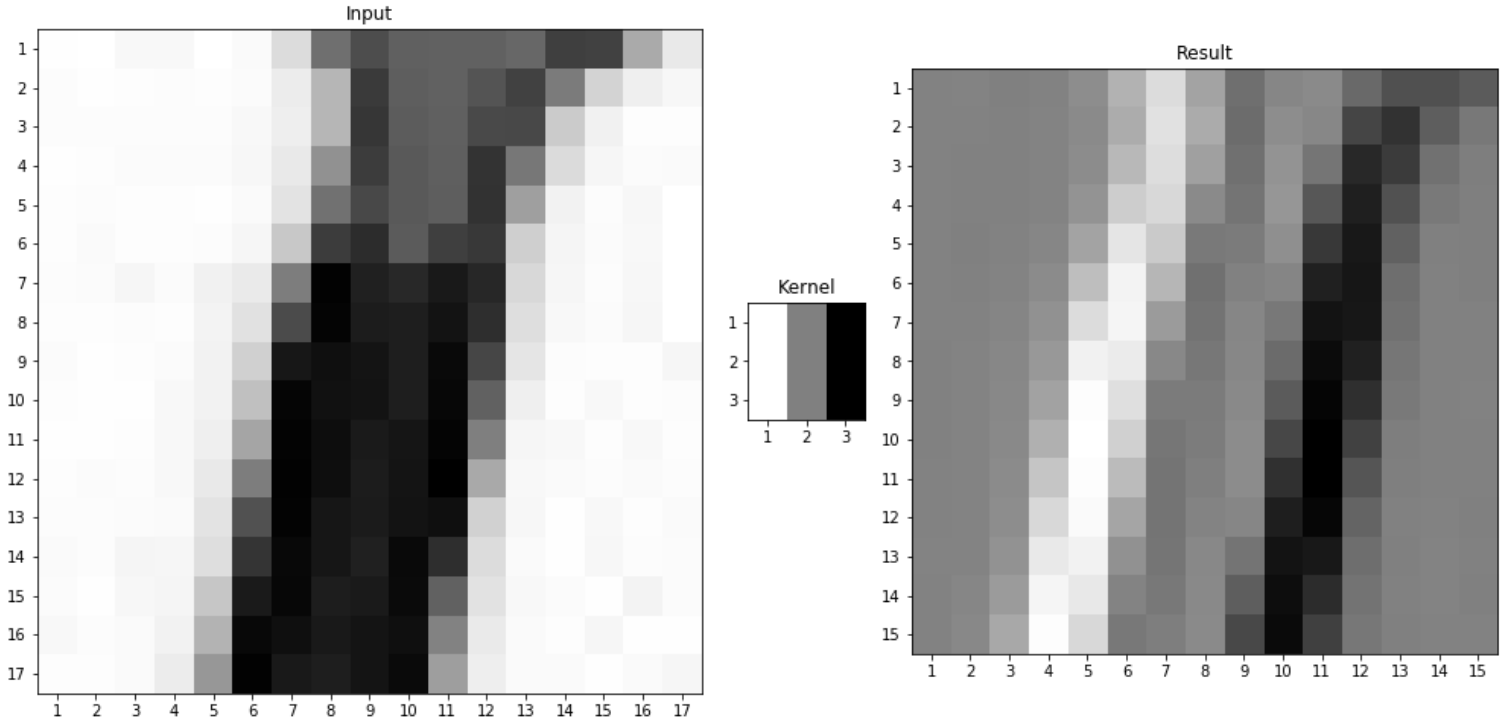
F = 7 => zero pad with 3

Convolution Visualizer



https://ezyang.github.io/convolution-visualizer/

Feature Extraction



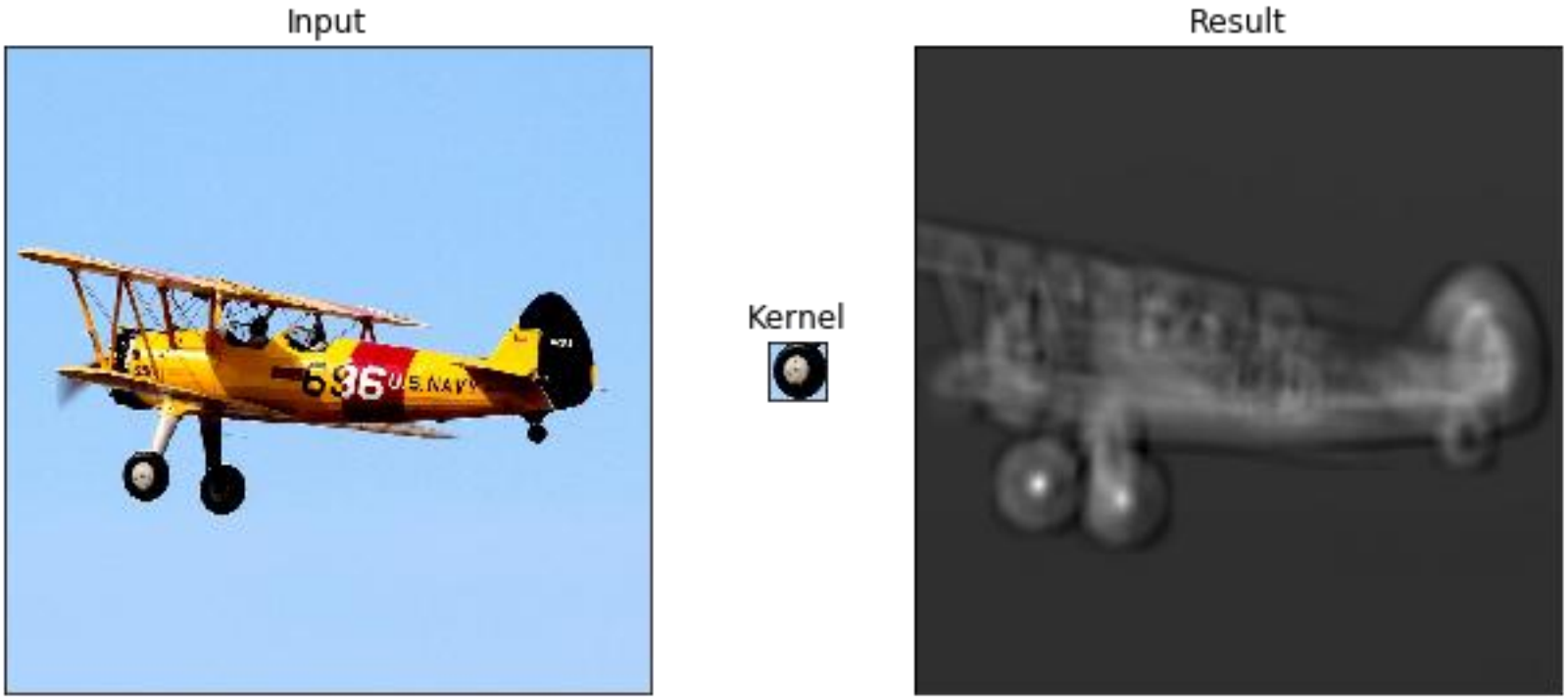
Source: https://towardsdatascience.com/visualizing-the-fundamentals-of-convolutional-neural-networks-6021e5b07f69

Convolution of an image 17x17 with an edge detector kernel 3x3.

Feature Extraction

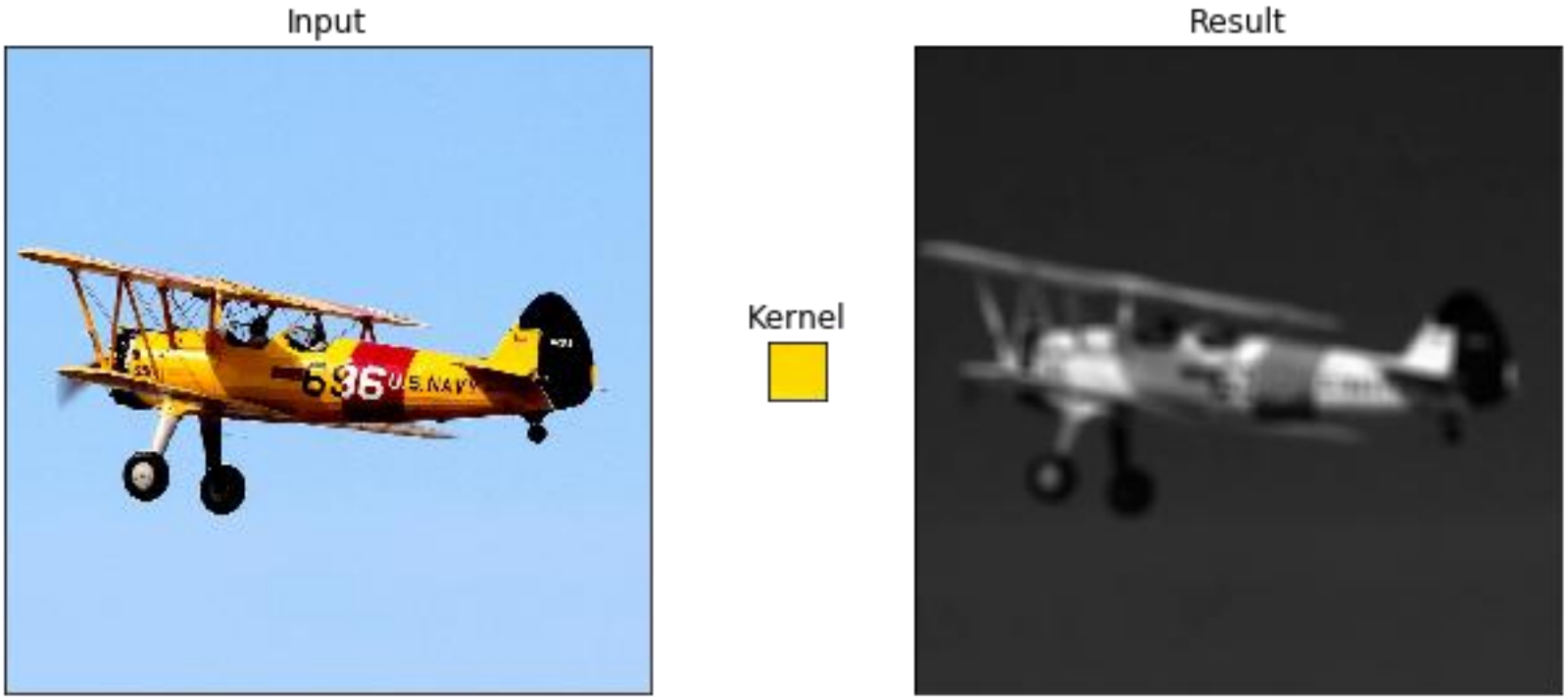
Source: https://towardsdatascience.com/visualizing-the-fundamentals-of-convolutional-neural-networks-6021e5b07f69

Feature Extraction



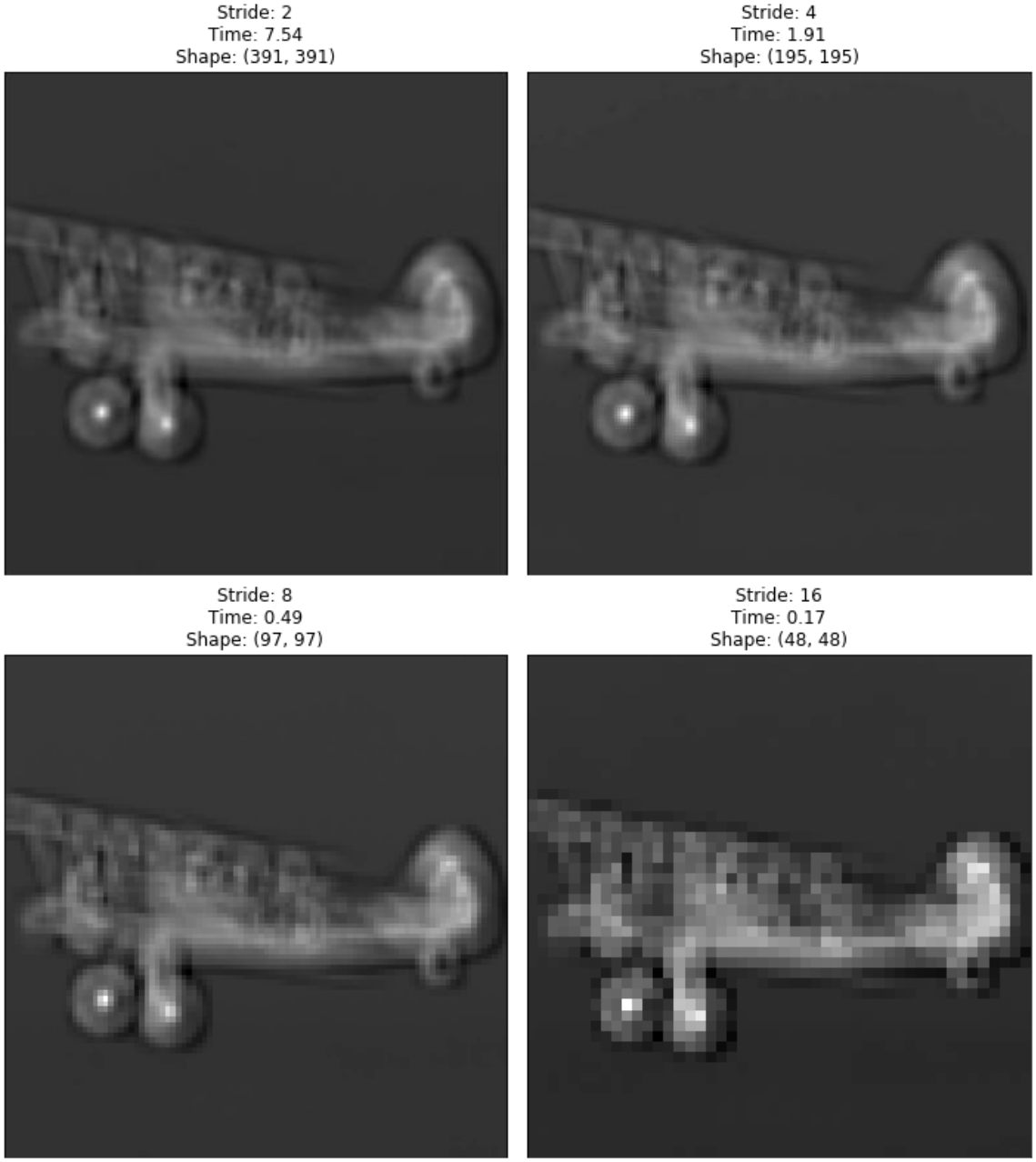
Source: https://towardsdatascience.com/visualizing-the-fundamentals-of-convolutional-neural-networks-6021e5b07f69

Feature Extraction



Source: https://towardsdatascience.com/visualizing-the-fundamentals-of-convolutional-neural-networks-6021e5b07f69

Feature Extraction

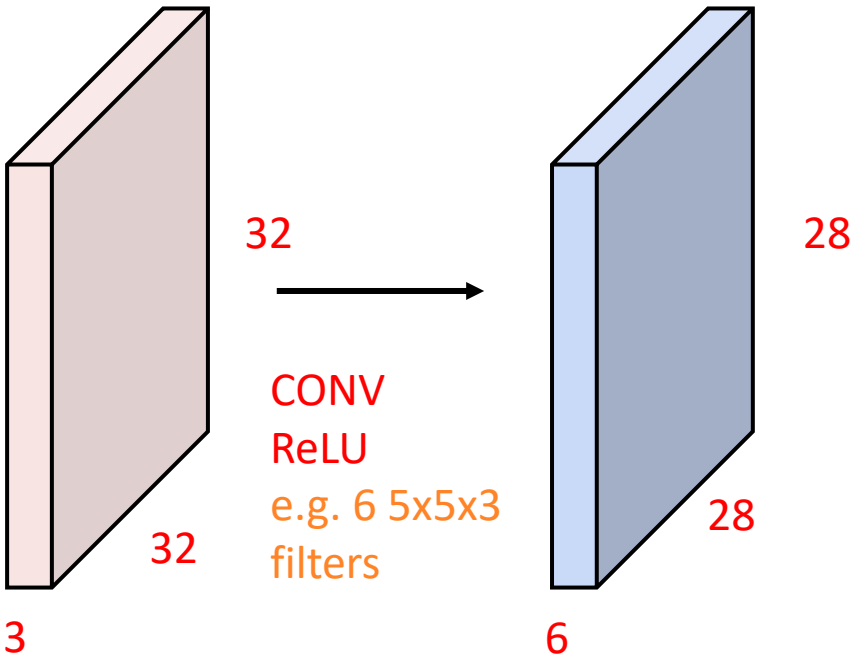


Source: https://towardsdatascience.com/visualizing-the-fundamentals-of-convolutional-neural-networks-6021e5b07f69

Convolutional Neural Network

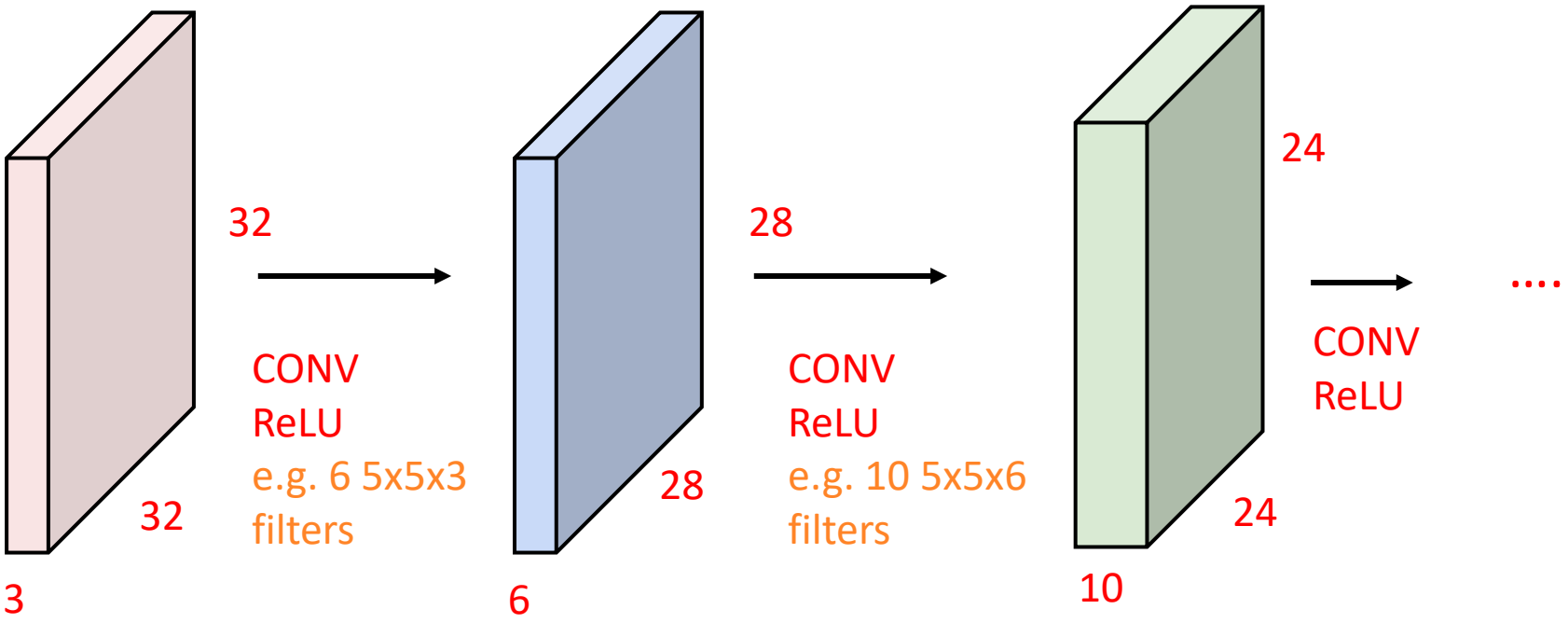
Convolutional Neural Network (CNN)

CNN is a sequence of **Convolution Layers** , interspersed with activation functions.

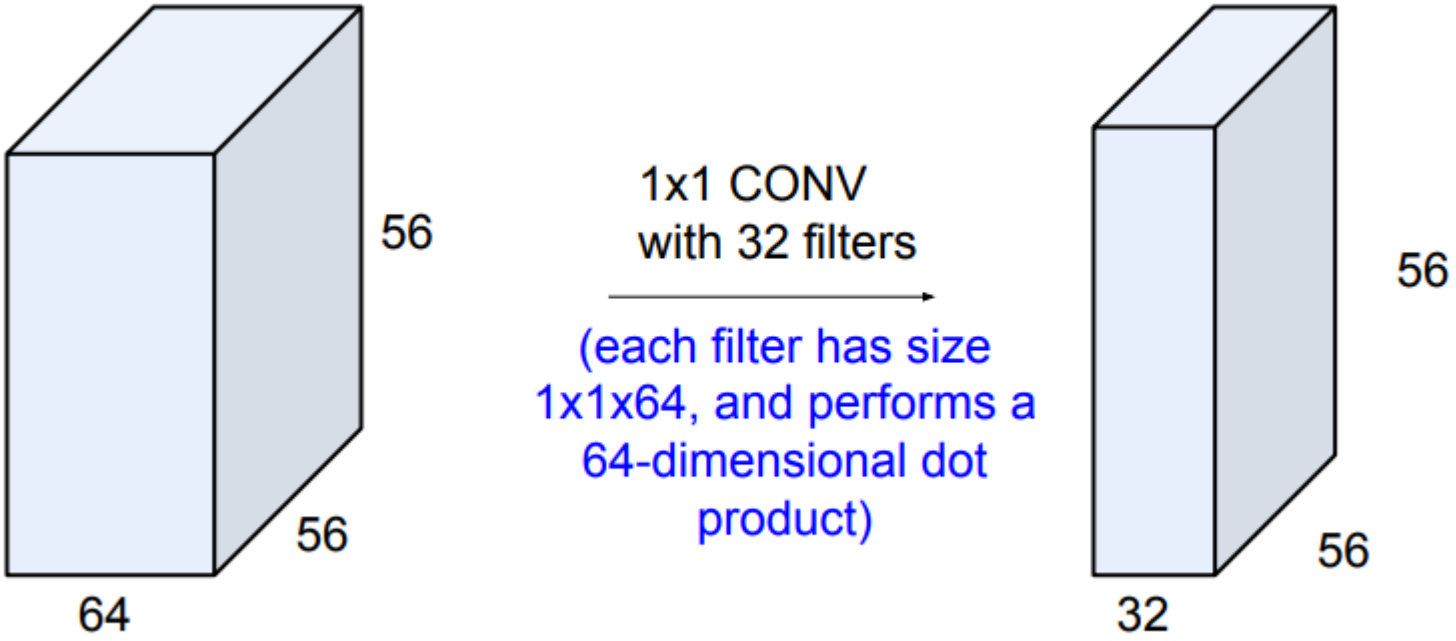


Convolutional Neural Network (CNN)

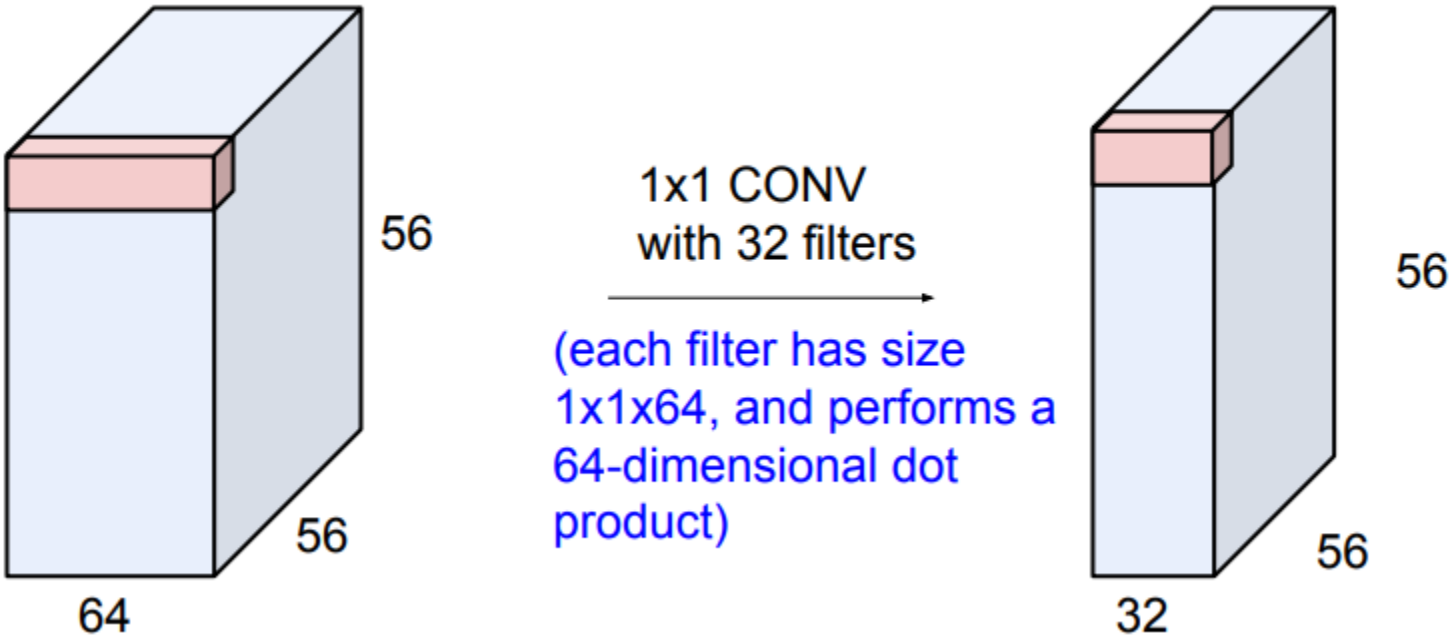
CNN is a sequence of **Convolution Layers** , interspersed with activation functions.



1 x 1 Convolution



1 x 1 Convolution



1x1 convolution kernel is meaningful! It computes the dot product over the channels.

Code Convolution Layer in Pytorch

CONV2D

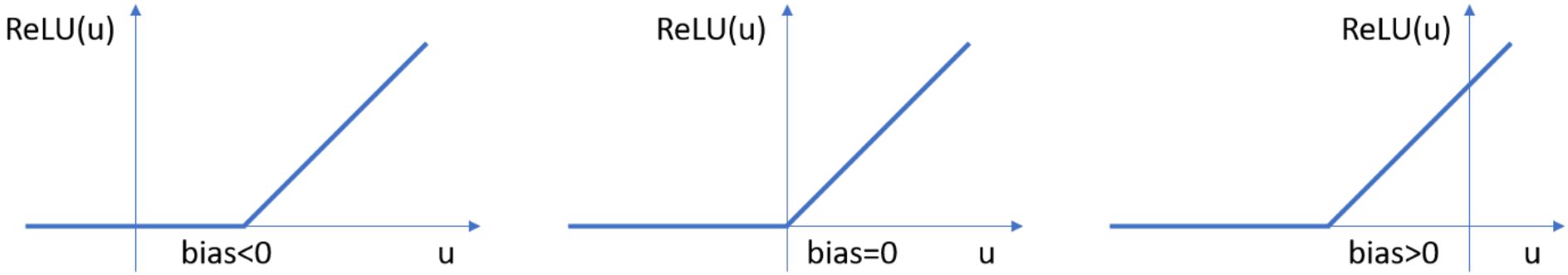
CLASS torch.nn.Conv2d(*in_channels, out_channels, kernel_size, stride=1, padding=0, dilation=1, groups=1, bias=True, padding_mode='zeros', device=None, dtype=None*) [SOURCE]

Examples

```
>>> # With square kernels and equal stride
>>> m = nn.Conv2d(16, 33, 3, stride=2)
>>> # non-square kernels and unequal stride and with padding
>>> m = nn.Conv2d(16, 33, (3, 5), stride=(2, 1), padding=(4, 2))
>>> # non-square kernels and unequal stride and with padding and dilation
>>> m = nn.Conv2d(16, 33, (3, 5), stride=(2, 1), padding=(4, 2), dilation=(3, 1))
>>> input = torch.randn(20, 16, 50, 100)
>>> output = m(input)
```

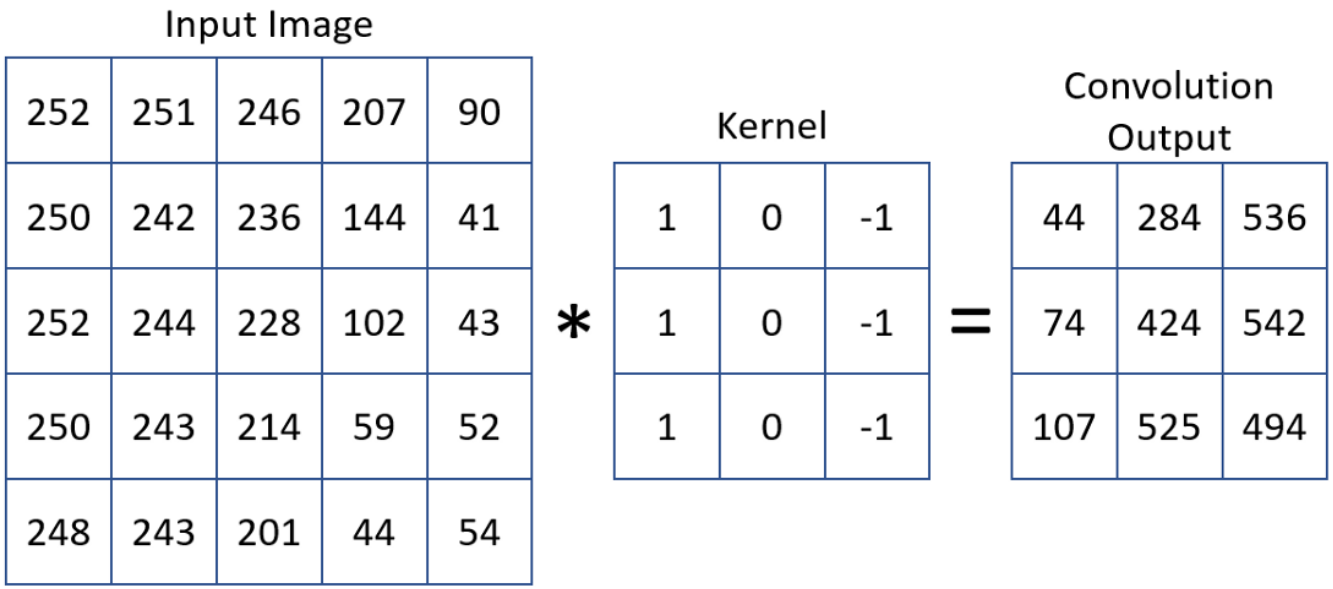
Non-linear Activations

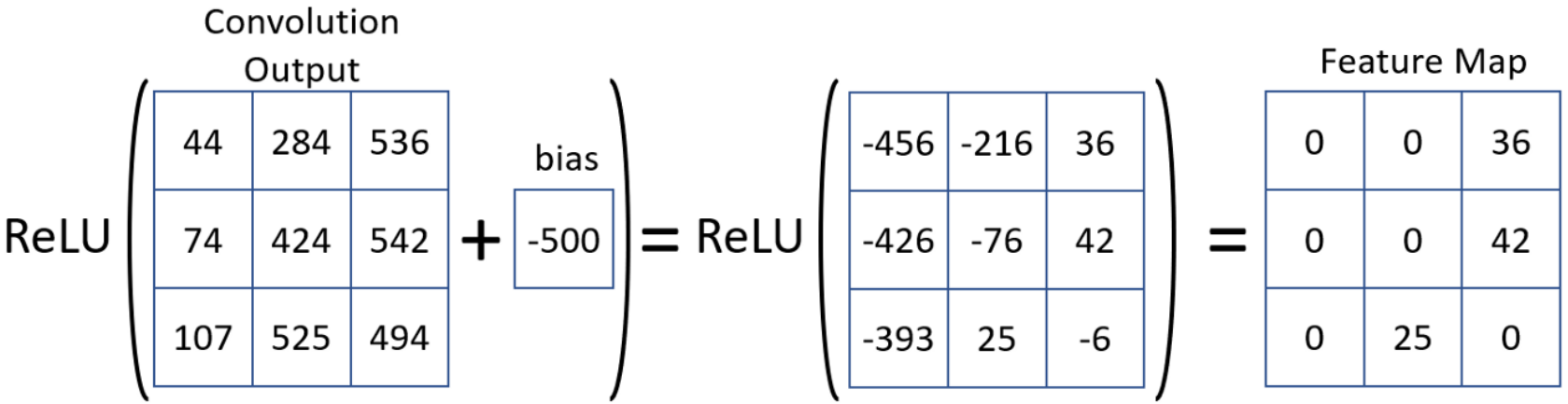
The bias affects the activation.



Source: https://towardsdatascience.com/visualizing-the-fundamentals-of-convolutional-neural-networks-6021e5b07f69

Non-linear Activations

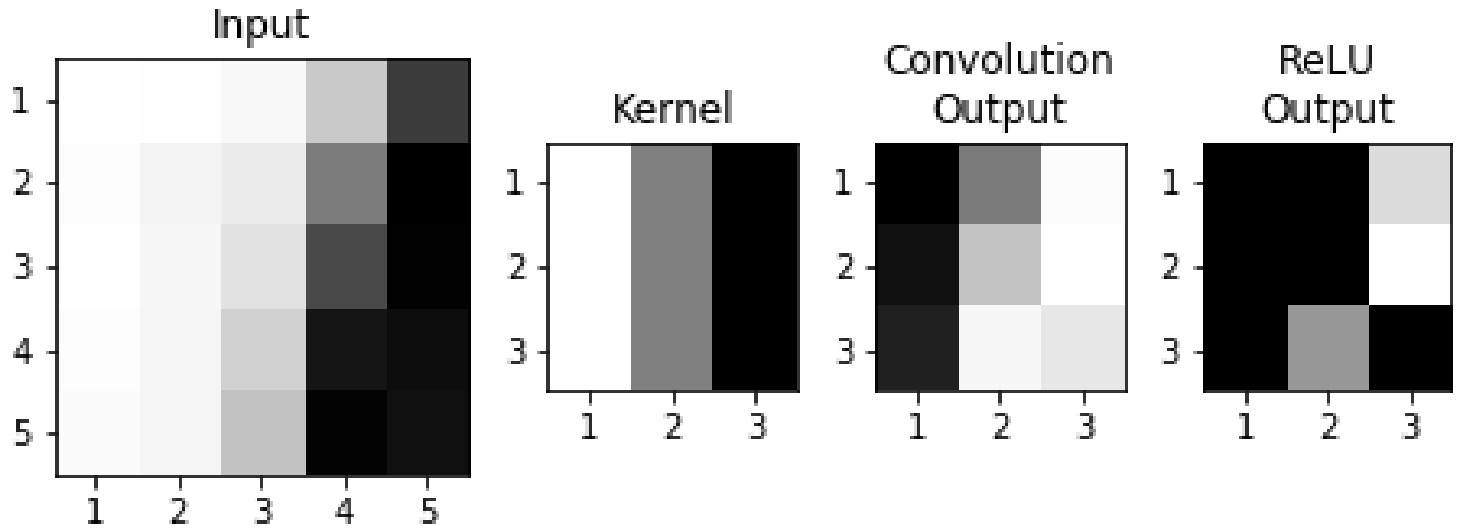




Source: https://towardsdatascience.com/visualizing-the-fundamentals-of-convolutional-neural-networks-6021e5b07f69

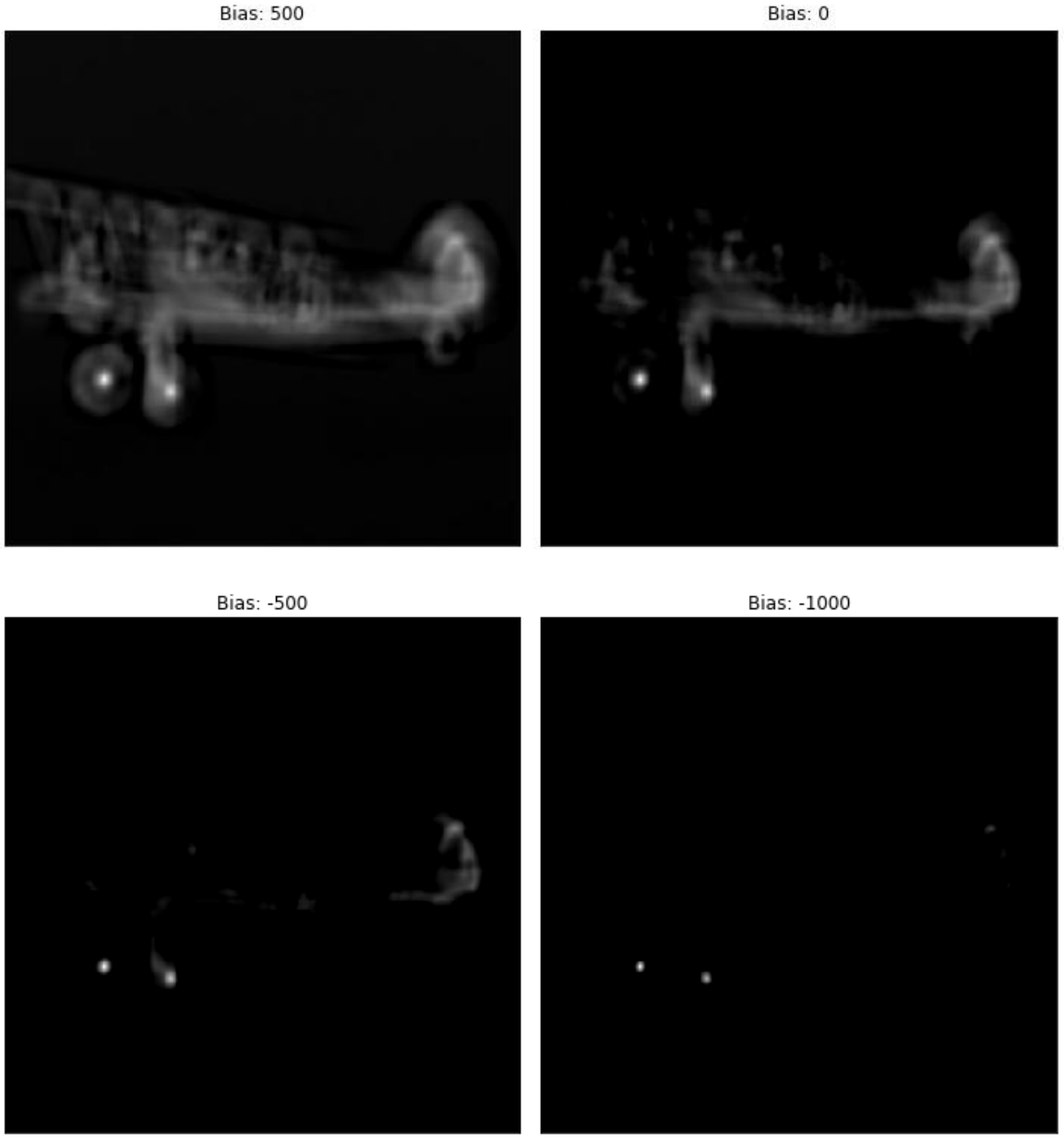
Non-linear Activations

ReLU activation function makes the negative inputs to be zero.



Source: https://towardsdatascience.com/visualizing-the-fundamentals-of-convolutional-neural-networks-6021e5b07f69

Non-linear Activations



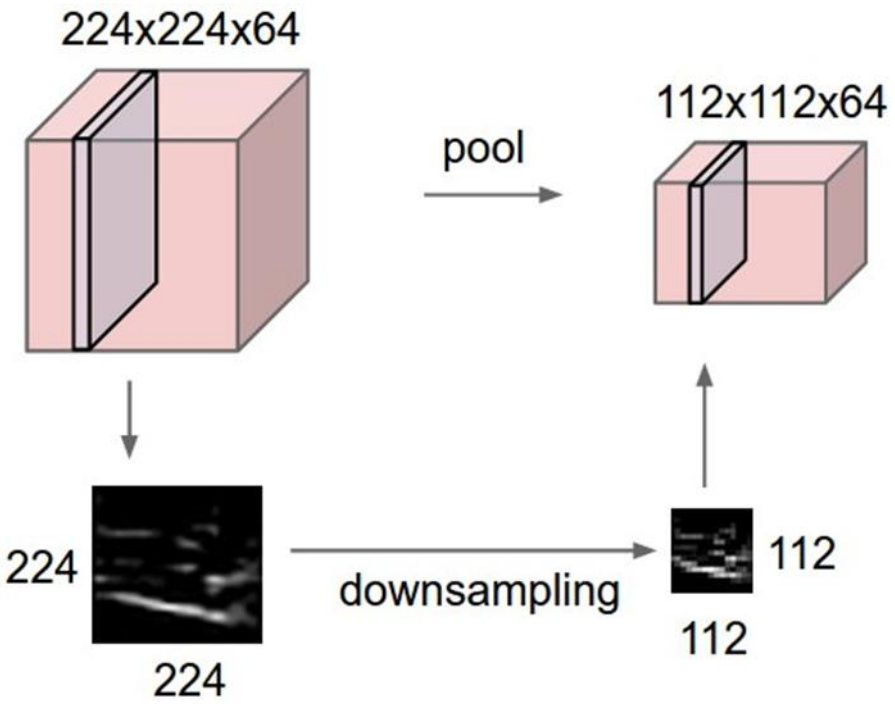
Source: https://towardsdatascience.com/visualizing-the-fundamentals-of-convolutional-neural-networks-6021e5b07f69

Pooling

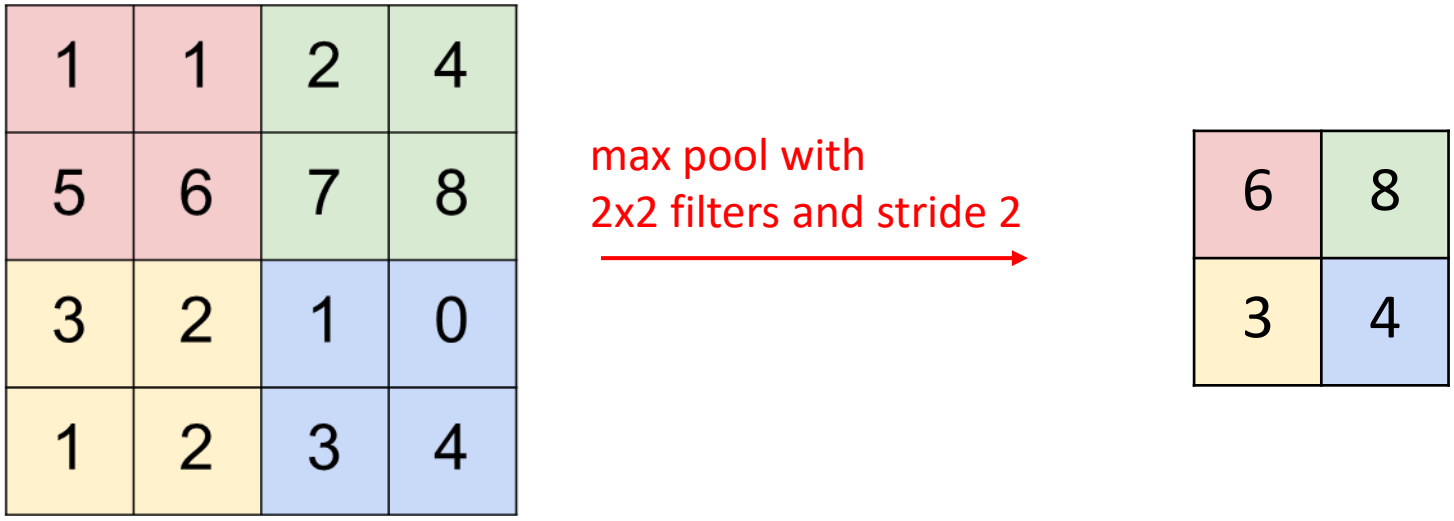
Pooling Layer

Pooling Layer is a down sampling strategy.

1. Construct better translationally invariant features.
2. Learn more compact features.

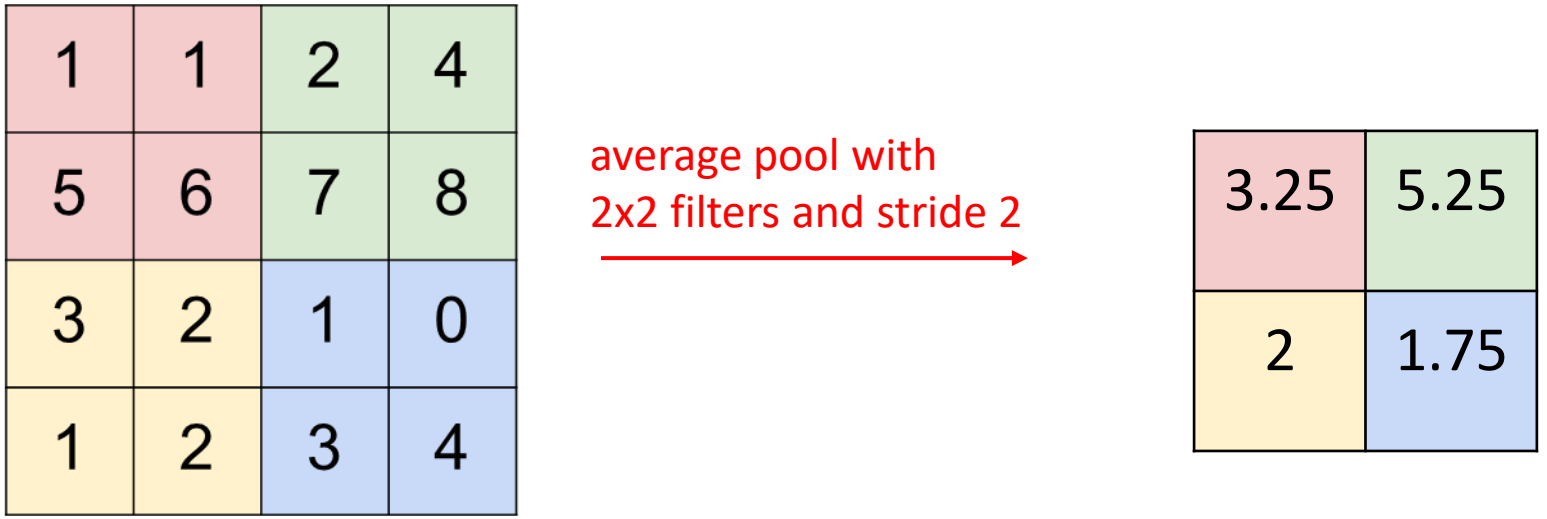


Max Pooling (Recommend)



Max Pooling Layer is robust to small perturbation.

Average Pooling



The Dimension After Pooling

Given a **D x M x N** tensor, if we apply the pooling operator with size **K x K** and Stride **P**, what are the dimensions of the output?

D x ((M - K)/P + 1) x ((N - K)/P + 1)

Code Pooling Layer in Pytorch

MAXPOOL2D

CLASS torch.nn.MaxPool2d(*kernel_size*, *stride=None*, *padding=0*, *dilation=1*, *return_indices=False*, *ceil_mode=False*) [SOURCE]

Examples:

```
>>> # pool of square window of size=3, stride=2
>>> m = nn.MaxPool2d(3, stride=2)
>>> # pool of non-square window
>>> m = nn.MaxPool2d((3, 2), stride=(2, 1))
>>> input = torch.randn(20, 16, 50, 32)
>>> output = m(input)
```

Receptive Fields

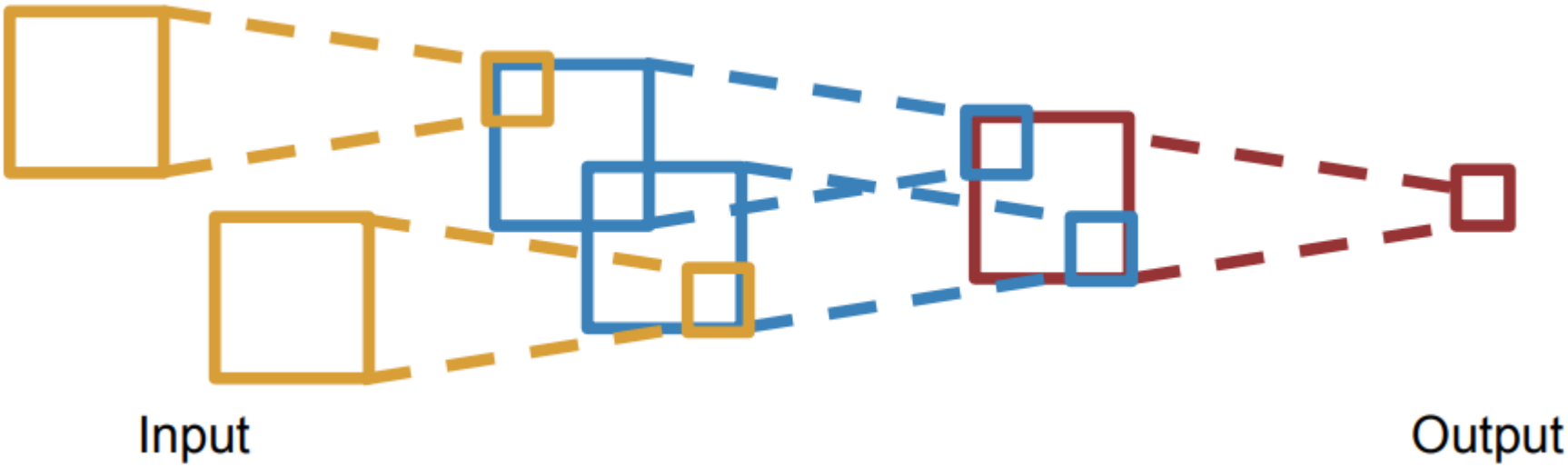
Receptive Fields

For convolution with kernel size **K**, each element in the output depends on a **K x K** receptive field in the input.

Input Output

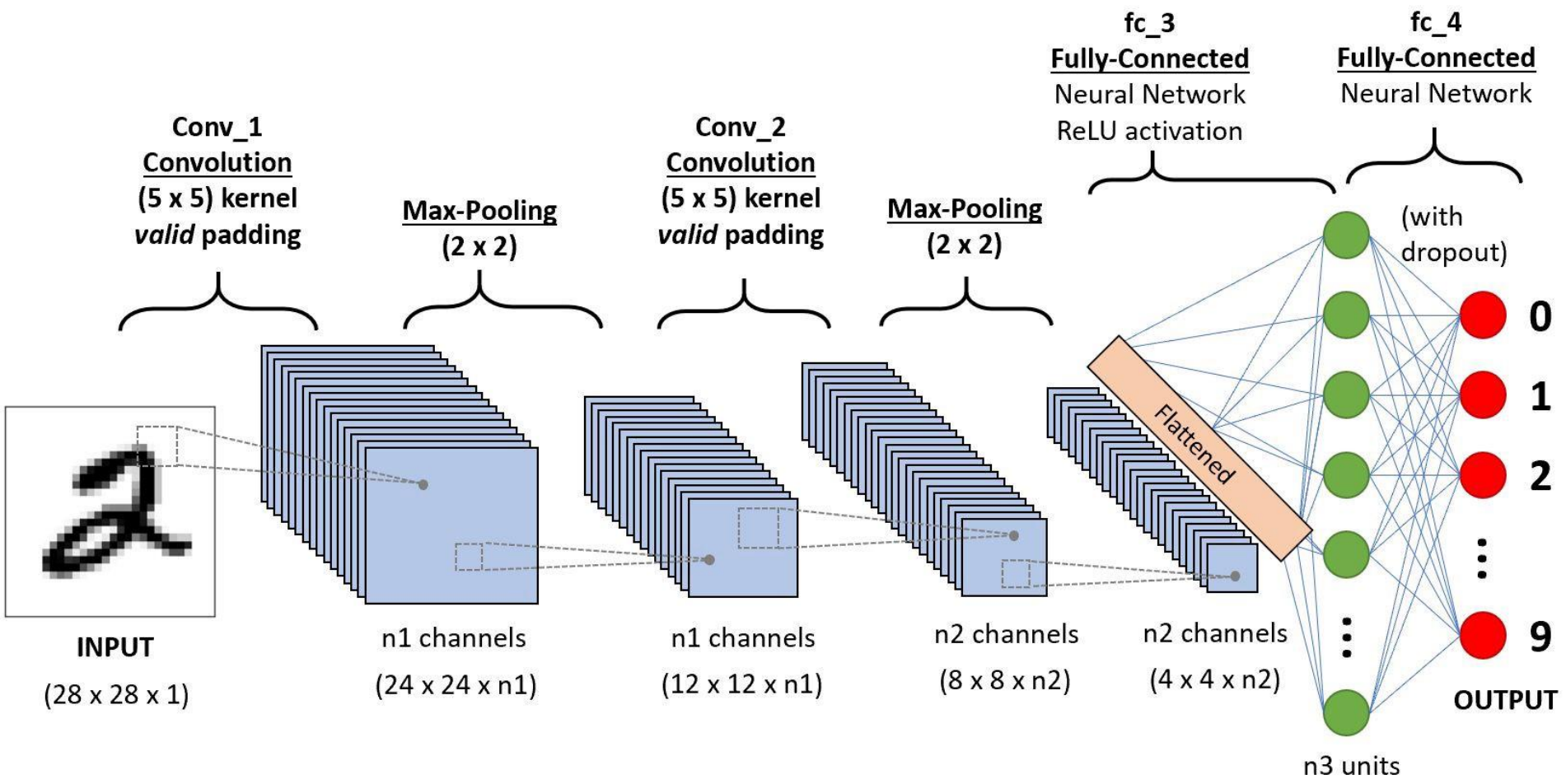
Multi-Layers

Each successive convolution contains multiple regions from the previous one.



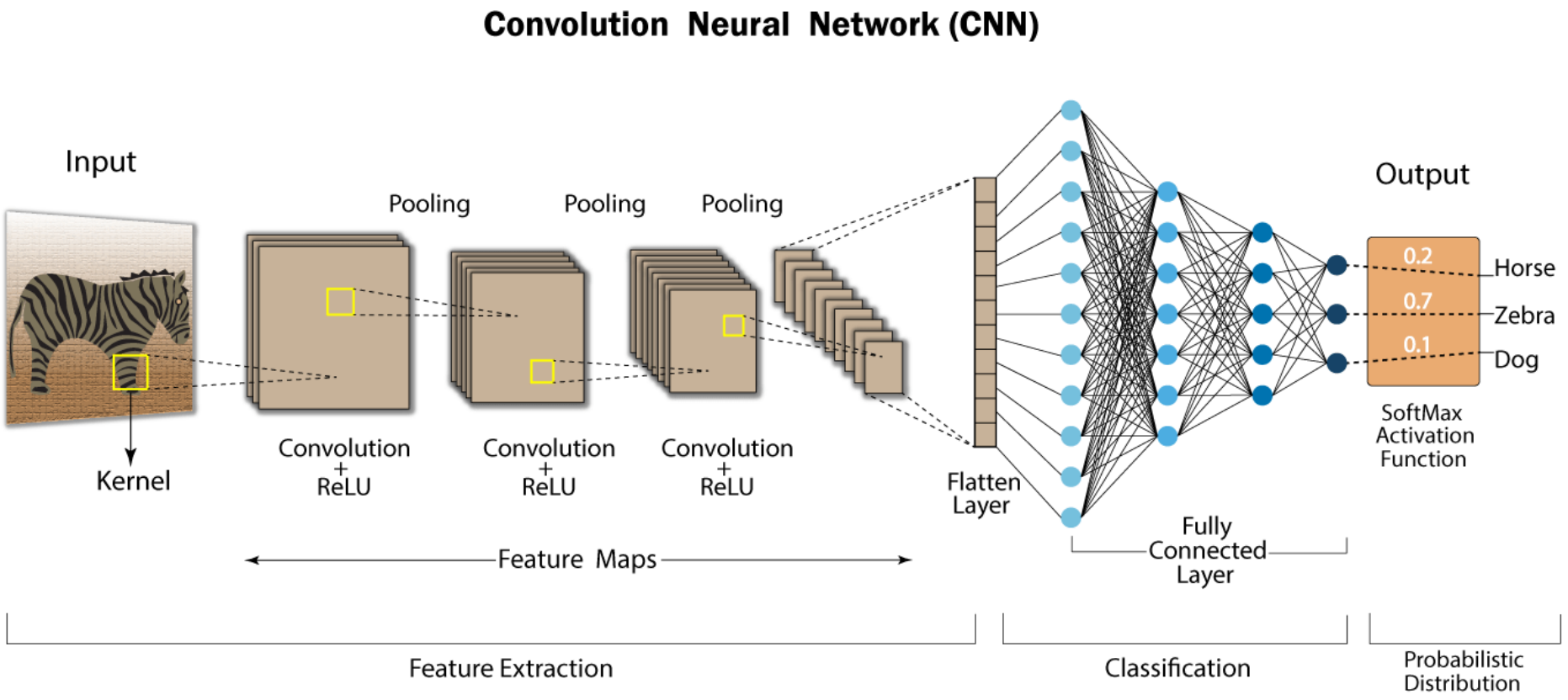
Suggested Reading: Computing receptive fields of convolutional neural network.

Examples of Convolutional Neural Network



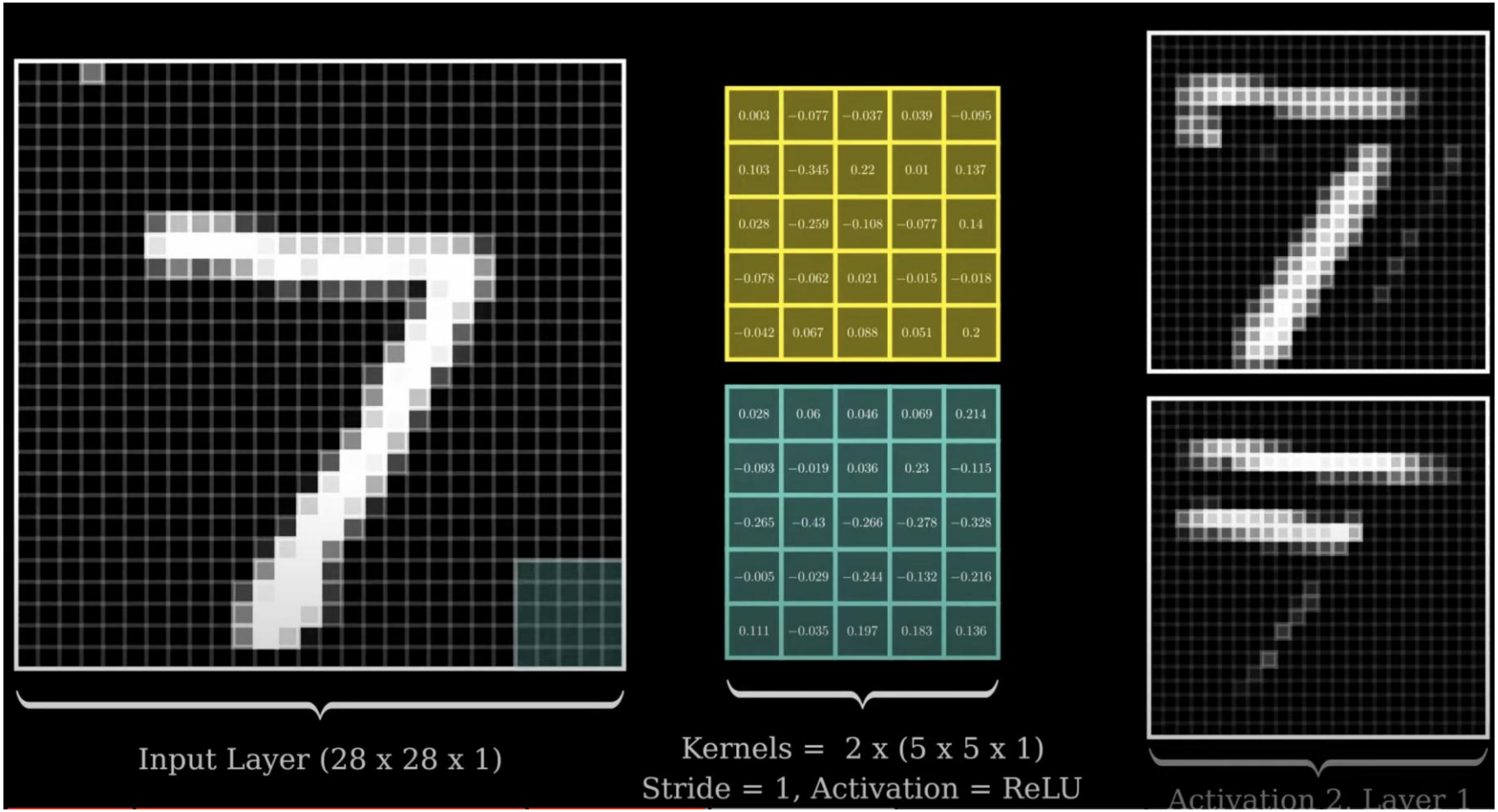
Source: https://www.analyticsvidhya.com/blog/2020/10/what-is-the-convolutional-neural-network-architecture/

Examples of Convolutional Neural Network



Source: https://developersbreach.com/convolution-neural-network-deep-learning/

Visualization of Convolutional Neural Network



[Visualizing Convolutional Neural Networks Layer by Layer]